

claude-windows / c6028e72-ebfd-4218-89a8-c4d7dd23be96

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c6028e72-ebfd-4218-89a8-c4d7dd23be96\subagents\workflows\wf_d251ad6c-472\agent-ad95e9f20fff6e5c1.jsonl`
- SHA-256: `257bdb76a0ffc374461c2cc0ecc234ece785dfdabb98ebc8d3db90ea2d141ec8`
- Source modified: `2026-06-08T08:41:55+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_d251ad6c-472`
- session_id: `c6028e72-ebfd-4218-89a8-c4d7dd23be96`
```

Transcript

1. user (2026-06-08T08:39:01.807Z)

Read the collector CANONICAL rally schema + validation + canonical CSV IO.
Files: C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/badminton.py,
C:/Users/avidu/Projects/sports-data-collector/src/core/schemas/base.py,
C:/Users/avidu/Projects/sports-data-collector/src/core/validator.py
(validate_badminton_rallies), C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py
(load_canonical_csv + canonical CSV read/write).
Determine, with exact field lists + anchors:

- The canonical rally record: required vs optional fields, types, defaults.
- Validation rules (overlap, ordering, zero-start handling, min/max duration, ending_reason vocabulary).
- The canonical CSV column set on read and on write.
- How ending_reason / sport / shots_count / outcome are represented (or not).
- What the canonical schema does NOT capture that the annotator + vendor-review CSVs provide (ending_reason taxonomy, last_shot_outcome, status/maturity, fractional rally numbers, missing end_time).

2. assistant (2026-06-08T08:39:04.344Z)

I'll systematically explore the sports-data-collector project to map out the canonical rally schema, validation rules, and CSV I/O. Let me start by reading the key files.

3. user (2026-06-08T08:39:04.750Z)

```
1 from pydantic import BaseModel, Field, model_validator
2 from typing import Optional
3
4 class BadmintonRally(BaseModel):
5     video_id: str
6     rally_number: int = Field(..., description="Chronological rally index in the match")
7     start_time: float = Field(..., description="Start of rally in seconds")
8     end_time: float = Field(..., description="End of rally in seconds")
9     score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10    score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11    shots_count: Optional[int] = Field(None, description="Number of strokes in the
rally")
12    ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13    last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15    @model_validator(mode='after')
16    def validate_timestamps(self) -> 'BadmintonRally':
```

```

17         if self.start_time < 0:
18             raise ValueError("start_time must be non-negative")
19         if self.end_time <= self.start_time:
20             raise ValueError("end_time must be strictly greater than start_time")
21         return self
22

```

4. user (2026-06-08T08:39:04.867Z)

```

1     from pydantic import BaseModel, Field
2     from typing import Optional
3     from enum import Enum
4
5     class SportType(str, Enum):
6         BADMINTON = "badminton"
7         TENNIS = "tennis"
8         TABLE_TENNIS = "tabletennis"
9         PICKLEBALL = "pickleball"
10        CRICKET = "cricket"
11
12    class LicenseType(str, Enum):
13        MIT = "MIT"
14        APACHE_2 = "Apache-2.0"
15        BSD = "BSD"
16        CC0 = "CC0"
17        CC_BY = "CC-BY"
18        CC_BY_NC = "CC-BY-NC"
19        CC_BY_NC_SA = "CC-BY-NC-SA"
20        PUBLIC_DOMAIN = "Public-Domain"
21        PROPRIETARY = "Proprietary"
22        UNKNOWN = "Unknown"
23
24    class CurationStatus(str, Enum):
25        STAGING = "staging"
26        CURATED = "curated"
27        REJECTED = "rejected"
28
29    class VideoMetadata(BaseModel):
30        video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
31        sport: SportType
32        video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
33        local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
34        license_type: LicenseType
35        license_url: Optional[str] = None
36        is_commercial: bool = Field(False, description="Flag indicating if the license
allows commercial use")
37        status: CurationStatus = CurationStatus.STAGING
38        match_name: str = Field(..., description="Name of the match/event")
39        fps: Optional[float] = Field(None, description="Frames per second")
40        resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
41

```

5. user (2026-06-08T08:39:05.012Z)

```

1     from typing import List, Tuple, Union
2     from src.core.config import load_config
3     from src.core.schemas.base import VideoMetadata, LicenseType
4     from src.core.schemas.badminton import BadmintonRally
5     from src.core.schemas.tennis import TennisPoint
6     from src.core.schemas.cricket import CricketDelivery
7     from src.core.schemas.pickleball import PickleballRally

```

```

8     from src.core.schemas.tabletennis import TableTennisRally
9
10    class DatasetValidator:
11        def __init__(self, config_path: str = "config/config.yaml"):
12            self.config_path = config_path
13            self.commercial_licenses = self._load_licenses_whitelist()
14
15        def _load_licenses_whitelist(self) -> List[str]:
16            # load_config returns {} on a missing/unreadable file, so the default list
17            # below is the single fallback (keep in sync with config.yaml).
18            config = load_config(self.config_path)
19            return config.get(
20                'commercial_licenses',
21                ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
22                "Proprietary"],
23            )
24
25        def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
26            """Determines if a license type is safe for commercial use based on the
27            configuration whitelist."""
28            val = license_type.value if isinstance(license_type, LicenseType) else
29            license_type
30            return val in self.commercial_licenses
31
32        def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
33        List[str]]:
34            """
35            Validates badminton rallies:
36            - Sorted chronologically.
37            - Start time < End time (handled by Pydantic, but double checked).
38            - No temporal overlap between sequential rallies.
39            """
40            errors = []
41            if not rallies:
42                return True, errors
43
44            # Sort by rally number to check progression
45            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
46
47            for i in range(len(sorted_rallies)):
48                r = sorted_rallies[i]
49                if r.start_time >= r.end_time:
50                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
51                    end_time ({r.end_time})")
52
53                if i > 0:
54                    prev = sorted_rallies[i - 1]
55                    if r.start_time < prev.end_time:
56                        errors.append(
57                            f"Rally overlap detected between Rally {prev.rally_number} "
58                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
59                            f"(starts at {r.start_time}s)"
60                        )
61
62            return len(errors) == 0, errors
63
64        def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
65        List[str]]:
66            errors = []
67            if not points:
68                return True, errors
69
70            sorted_pts = sorted(points, key=lambda p: p.point_number)
71            for i in range(len(sorted_pts)):
72                pt = sorted_pts[i]

```

```

67         if pt.start_time >= pt.end_time:
68             errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70         if i > 0:
71             prev = sorted_pts[i - 1]
72             if pt.start_time < prev.end_time:
73                 errors.append(
74                     f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                     f"and Point {pt.point_number} (starts {pt.start_time}s)"
76                 )
77         return len(errors) == 0, errors
78
79     def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80         errors = []
81         if not deliveries:
82             return True, errors
83
84         # Sort by over, then ball within over
85         sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86         for i in range(len(sorted_delivs)):
87             d = sorted_delivs[i]
88             if d.start_time >= d.end_time:
89                 errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")
90
91             if i > 0:
92                 prev = sorted_delivs[i - 1]
93                 if d.start_time < prev.end_time:
94                     errors.append(
95                         f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
96                         f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
97                     )
98         return len(errors) == 0, errors
99
100     def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
101         """Validates pickleball rallies: positive durations and no temporal overlap
102         between chronologically-ordered rallies (same contract as badminton)."""
103         errors = []
104         if not rallies:
105             return True, errors
106
107         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
108         for i in range(len(sorted_rallies)):
109             r = sorted_rallies[i]
110             if r.start_time >= r.end_time:
111                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
112
113             if i > 0:
114                 prev = sorted_rallies[i - 1]
115                 if r.start_time < prev.end_time:
116                     errors.append(
117                         f"Rally overlap detected between Rally {prev.rally_number} "
118                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
119                         f"(starts at {r.start_time}s)"
120                     )
121         return len(errors) == 0, errors
122
123     def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:

```

```

124         """Validates table-tennis rallies: positive durations and no temporal overlap
125         between chronologically-ordered rallies (same contract as badminton)."""
126         errors = []
127         if not rallies:
128             return True, errors
129
130         sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
131         for i in range(len(sorted_rallies)):
132             r = sorted_rallies[i]
133             if r.start_time >= r.end_time:
134                 errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
135
136             if i > 0:
137                 prev = sorted_rallies[i - 1]
138                 if r.start_time < prev.end_time:
139                     errors.append(
140                         f"Rally overlap detected between Rally {prev.rally_number} "
141                         f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
142                         f"(starts at {r.start_time}s)"
143                     )
144         return len(errors) == 0, errors
145

```

6. user (2026-06-08T08:39:05.078Z)

```

1     """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3     Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4     ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5     """
6     from typing import Any, Optional
7
8
9     def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10         """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11         if value is None:
12             return default
13         s = str(value).strip()
14         if not s:
15             return default
16         try:
17             return int(float(s))
18         except (TypeError, ValueError):
19             return default
20
21
22     def safe_float_required(value: Any) -> float:
23         """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24         if value is None:
25             raise ValueError("missing required numeric value")
26         s = str(value).strip()
27         if not s:
28             raise ValueError("missing required numeric value")
29         return float(s)
30
31
32     def parse_time_string(time_str: Any) -> float:
33         """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34         if not time_str:
35             return 0.0
36         s = str(time_str).strip()

```

```

37         if ":" in s:
38             parts = s.split(":")
39             try:
40                 if len(parts) == 3:
41                     return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                 if len(parts) == 2:
43                     return float(parts[0]) * 60 + float(parts[1])
44             except ValueError:
45                 return 0.0
46         try:
47             return float(s)
48         except ValueError:
49             return 0.0
50

```

7. assistant (2026-06-08T08:39:07.006Z)

Now let me search for the canonical CSV I/O functions in parsing.py:

8. user (2026-06-08T08:39:08.053Z)

No matches found

9. user (2026-06-08T08:39:08.201Z)

49 C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py

10. assistant (2026-06-08T08:39:10.331Z)

The parsing.py file I read is only 49 lines and contains helper functions. Let me search for canonical CSV handling across the project:

11. user (2026-06-08T08:39:11.099Z)

```

C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\base_parser.py
C:\Users\avidu\Projects\sports-data-collector\src\core\downloader.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\ingest.py
C:\Users\avidu\Projects\sports-data-collector\src\core\config.py
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py
C:\Users\avidu\Projects\sports-data-collector\src\core\validator.py
C:\Users\avidu\Projects\sports-data-collector\src\crawlers\badminton_crawler.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\__init__.py
C:\Users\avidu\Projects\sports-data-collector\src\core\db.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\src\reporting\__init__.py
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py

```

12. user (2026-06-08T08:39:11.182Z)

Found 4 files

```

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py

```

13. assistant (2026-06-08T08:39:13.121Z)

Let me check the CLI curate file and the CVAT parser for canonical CSV handling:

14. user (2026-06-08T08:39:13.516Z)

```
1      """Adapter: CVAT rally exports -> our canonical per-rally records.
2
3      Why this exists
4      -----
5      Our annotation brief (`docs/annotation/ROORA_BRIEF.md`) asks vendors for a
6      per-rally **CSV** that imports directly via `curate import-local`. A vendor may
7      instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
8      boxes, with the rally fields (`rally_number`, `shots_count`,
9      `ending_reason` ...) carried as CVAT **object attributes** on each box. The box
10     geometry is irrelevant to us ? it is just a carrier the rater attached the rally
11     metadata to.
12
13     This module is the ingestion-side adapter that absorbs that vendor format so the
14     modeling repo never has to. It does two things, both deliberately on *our* side
15     so we fix mechanical problems rather than renegotiating them:
16
17     1. **Recompute time from the frame range.** A rally's interval is taken from the
18         first/last *frame* a box for that rally appears on:
19         `seconds = frame / fps` at the video's true fps. We **ignore** any
20         `start_time`/`end_time` attributes the vendor wrote ? a CVAT job that
21         samples every `step` frames and assumes 30 fps yields systematically wrong
22         times (we have seen `start_time == frame / (step * 30)`, i.e. off by the
23         sampling step at the wrong fps). The frame index is reliable; the vendor's
24         seconds are not.
25
26     2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
27         zero-duration rallies, ending_reason values outside the agreed set, and
28         too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
29         rally boundary would poison ground truth, so that stays a human/vendor call.
30
31     Supported input: the single-file CVAT XML export, in either **image mode**
32     (`<image><box>...`) or **track/interpolation mode** (`<track><box frame=...>`).
33     The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
34     later if a vendor sends only those.
35     """
36     from __future__ import annotations
37
38     import os
39     import xml.etree.ElementTree as ET
40     from collections import Counter
41     from dataclasses import dataclass, field
42     from typing import Any, Dict, List, Optional, Tuple
43
44     from src.core.parsing import safe_int
45     from src.core.downloader import probe_resolution_fps
46
47     # The agreed cross-sport ending_reason vocabulary for net-separated sports.
48     # Kept here as a soft check only (a warning, never a hard failure) so this
49     # adapter does not depend on the annotation-spec wording.
50     DEFAULT_ALLOWED_REASONS = {
51         "winner",
52         "forced_error",
53         "unforced_error",
54         "service_fault",
55         "let",
56         "other",
57     }
58
59     # The observational outcome of the last shot ? orthogonal to ending_reason
60     # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61     DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
```

```

62
63 # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64 # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65 _ATTR_ALIASES = {
66     "rally_number": ("rally_number",),
67     "shots_count": ("shots_count",),
68     "ending_reason": ("ending_reason",),
69     "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70     "score_before": ("score_before", "score_before (a-b)"),
71     "score_after": ("score_after", "score_after (a-b)"),
72     "notes": ("notes", "note"),
73 }
74
75 CANONICAL_COLUMNS = [
76     "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
77     "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
78 ]
79
80
81 @dataclass
82 class RallyRecord:
83     """One rally recovered from the CVAT export, time re-derived from frames."""
84     rally_number: int
85     start_time: float          # seconds, = frame_min / fps
86     end_time: float           # seconds, = frame_max / fps (see padding note)
87     shots_count: Optional[int]
88     ending_reason: Optional[str]
89     last_shot_outcome: Optional[str]
90     score_before: Optional[str]
91     score_after: Optional[str]
92     notes: Optional[str]
93     frame_min: int
94     frame_max: int
95     n_boxes: int
96     vendor_start_time: Optional[str] = None    # the (discarded) attribute value
97     vendor_end_time: Optional[str] = None
98
99     @property
100     def duration(self) -> float:
101         return self.end_time - self.start_time
102
103
104 @dataclass
105 class Issue:
106     """A defect found in the delivery. ``blocker`` issues stop a clean import."""
107     severity: str    # 'blocker' | 'high' | 'medium' | 'low'
108     code: str
109     message: str
110     rally_number: Optional[int] = None
111
112
113 @dataclass
114 class ConversionResult:
115     records: List[RallyRecord]
116     issues: List[Issue]
117     fps: float
118     step: int
119     meta: Dict[str, Any] = field(default_factory=dict)
120
121     @property
122     def n_blockers(self) -> int:
123         return sum(1 for i in self.issues if i.severity == "blocker")
124

```



```

125
126 def _mmss(seconds: float) -> str:
127     """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
128     m = int(seconds // 60)
129     s = seconds - m * 60
130     return f"{m}:{s:06.3f}"
131
132
133 def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
134     for alias in _ATTR_ALIASES[canonical]:
135         if alias in attrs and attrs[alias] != "":
136             return attrs[alias]
137     return None
138
139
140 def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
141     """Parse a CVAT XML export into (meta, boxes).
142
143     ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
144     entry per visible box, from either image-mode or track-mode exports.
145     ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
146     """
147     if not os.path.exists(xml_path):
148         raise FileNotFoundError(f"CVAT export not found: {xml_path}")
149
150     root = ET.parse(xml_path).getroot()
151     meta: Dict[str, Any] = {}
152
153     job = root.find("./job")
154     if job is not None:
155         stop = job.findtext("stop_frame")
156         if stop is not None and stop.strip().isdigit():
157             meta["stop_frame"] = int(stop.strip())
158         ff = job.findtext("frame_filter") or ""
159         if "step=" in ff:
160             try:
161                 meta["step"] = int(ff.split("step=")[1].split())[0])
162             except (ValueError, IndexError):
163                 pass
164
165     def _attrs(el: ET.Element) -> Dict[str, str]:
166         out: Dict[str, str] = {}
167         for a in el.findall("attribute"):
168             out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
169         return out
170
171     boxes: List[Dict[str, Any]] = []
172
173     # Image mode: <image id=frame> ... <box>...</box>
174     for img in root.findall("image"):
175         try:
176             frame = int(img.get("id"))
177         except (TypeError, ValueError):
178             continue
179         for box in img.findall("box"):
180             boxes.append({"frame": frame, "attrs": _attrs(box)})
181
182     # Track mode: <track> ... <box frame=N outside=0>...</box>
183     for track in root.findall("track"):
184         for box in track.findall("box"):
185             if box.get("outside") == "1":
186                 continue # a track's terminating keyframe carries no real label
187             try:
188                 frame = int(box.get("frame"))
189             except (TypeError, ValueError):

```

```

190         continue
191         boxes.append({"frame": frame, "attrs": _attrs(box)})
192
193     return meta, boxes
194
195
196 def _infer_step(frames: List[int]) -> int:
197     """Most common positive gap between consecutive distinct sampled frames."""
198     uniq = sorted(set(frames))
199     gaps = [b - a for a, b in zip(uniq, uniq[1:]) if b - a > 0]
200     if not gaps:
201         return 1
202     return Counter(gaps).most_common(1)[0][0]
203
204
205 def build_records(boxes: List[Dict[str, Any]], fps: float) -> List[RallyRecord]:
206     """Group boxes by rally_number and derive each rally's frame range -> seconds."""
207     if fps is None or fps <= 0:
208         raise ValueError("fps must be a positive number to convert frames to seconds")
209
210     by_rally: Dict[int, List[Dict[str, Any]]] = {}
211     for b in boxes:
212         rn = safe_int(_attr_lookup(b["attrs"], "rally_number"), None)
213         if rn is None:
214             continue
215         by_rally.setdefault(rn, []).append(b)
216
217     records: List[RallyRecord] = []
218     for rn in sorted(by_rally):
219         items = by_rally[rn]
220         frames = sorted(b["frame"] for b in items)
221         attrs = items[0]["attrs"]
222         fmin, fmax = frames[0], frames[-1]
223         records.append(RallyRecord(
224             rally_number=rn,
225             start_time=fmin / fps,
226             end_time=fmax / fps,
227             shots_count=safe_int(_attr_lookup(attrs, "shots_count"), None),
228             ending_reason=_attr_lookup(attrs, "ending_reason"),
229             last_shot_outcome=_attr_lookup(attrs, "last_shot_outcome"),
230             score_before=_attr_lookup(attrs, "score_before"),
231             score_after=_attr_lookup(attrs, "score_after"),
232             notes=_attr_lookup(attrs, "notes"),
233             frame_min=fmin,
234             frame_max=fmax,
235             n_boxes=len(frames),
236             vendor_start_time=attrs.get("start_time"),
237             vendor_end_time=attrs.get("end_time"),
238         ))
239     return records
240
241
242 def detect_issues(records: List[RallyRecord],
243                  stop_frame: Optional[int] = None,
244                  allowed_reasons: Optional[set] = None,
245                  allowed_outcomes: Optional[set] = None,
246                  max_gap_s: float = 15.0) -> List[Issue]:
247     """Find the defects that block or degrade a clean ground-truth import.
248
249     Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
250     slack of ~0.5s is acceptable for this pipeline because the downstream
251     highlight compiler pads every clip on output (badminton-highlight-indexer,
252     backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
253     absorbs the sampling uncertainty when highlights are trimmed.
254     """

```



```

318         f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
319         f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
320         f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
321         r.rally_number))
322     return issues
323
324
325 def convert(xml_path: str,
326            fps: Optional[float] = None,
327            video_path: Optional[str] = None,
328            step: Optional[int] = None,
329            allowed_reasons: Optional[set] = None) -> ConversionResult:
330     """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.
331
332     ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
333     the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
334     that silent guess is exactly the bug this adapter exists to undo.
335     """
336     meta, boxes = parse_cvat_boxes(xml_path)
337
338     if fps is None and video_path:
339         _, _, probed = probe_resolution_fps(video_path)
340         fps = probed
341     if fps is None or fps <= 0:
342         raise ValueError(
343             "Could not determine fps. Pass fps=... or a readable video_path so "
344             "frames can be converted to seconds (seconds = frame / fps).")
345
346     if step is None:
347         step = meta.get("step") or _infer_step([b["frame"] for b in boxes])
348
349     records = build_records(boxes, fps)
350     issues = detect_issues(records,
351                            stop_frame=meta.get("stop_frame"),
352                            allowed_reasons=allowed_reasons)
353     return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)
354
355
356 def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
357     """Write records as the canonical per-rally CSV that ``import-local`` reads."""
358     import csv
359     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
360     with open(out_path, "w", newline="", encoding="utf-8") as f:
361         w = csv.writer(f)
362         w.writerow(CANONICAL_COLUMNS)
363         for r in sorted(records, key=lambda x: x.rally_number):
364             w.writerow([
365                 r.rally_number,
366                 _mmss(r.start_time), _mmss(r.end_time),
367                 f"{r.start_time:.3f}", f"{r.end_time:.3f}",
368                 r.shots_count if r.shots_count is not None else "",
369                 r.ending_reason or "",
370                 r.last_shot_outcome or "",
371                 r.score_before or "", r.score_after or "", r.notes or "",
372             ])
373
374
375 def render_issue_report(result: ConversionResult, source: str) -> str:
376     """A human-readable issue report (for review / sending back to the vendor)."""
377     lines = [
378         f"CVAT rally conversion report ? {source}",
379         f"  fps used           : {result.fps:.3f}",
380         f"  sampling step       : {result.step} frames (~{result.step / result.fps:.2f}s)",

```

```

381         f" rallies parsed : {len(result.records)}",
382         f" issues          : {len(result.issues)} ({result.n_blockers} blocker)",
383         "",
384     ]
385     if not result.issues:
386         lines.append("No issues detected.")
387     else:
388         for sev in ("blocker", "high", "medium", "low"):
389             group = [i for i in result.issues if i.severity == sev]
390             if not group:
391                 continue
392             lines.append(f"[{sev.upper()}]")
393             for i in group:
394                 lines.append(f" - ({i.code}) {i.message}")
395             lines.append("")
396     return "\n".join(lines)
397
398
399 # ----- #
400 # Validation reporting ? used by `curate validate-delivery`. #
401 # These turn a (records, issues) pair into the artifacts a human reviews: #
402 # a machine summary (JSON), a human report (md), and an annotatable sheet. #
403 # ----- #
404
405 def _parse_secs(value: Optional[str]) -> Optional[float]:
406     """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
407     s = (value or "").strip()
408     if not s:
409         return None
410     if ":" in s:
411         try:
412             total = 0.0
413             for part in s.split(":"):
414                 total = total * 60 + float(part)
415             return total
416         except ValueError:
417             return None
418     try:
419         return float(s)
420     except ValueError:
421         return None
422
423
424 def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
425     """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
426     just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
427     fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
428     CSV carries no frame info ? only the frame-derived checks become inapplicable.)
429     """
430     import csv as _csv
431     if not os.path.exists(csv_path):
432         raise FileNotFoundError(f"CSV not found: {csv_path}")
433     records: List[RallyRecord] = []
434     with open(csv_path, newline="", encoding="utf-8") as f:
435         reader = _csv.DictReader(f)
436         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
437 [])]
438         for raw in reader:
439             row = {(k or "").strip().lower(): v for k, v in raw.items()}
440             rn = safe_int(row.get("rally_number"), None)
441             if rn is None:
442                 continue
443             start = _parse_secs(row.get("start_time"))
444             if start is None:
445                 start = _parse_secs(row.get("start_mmss"))

```

```

445         end = _parse_secs(row.get("end_time"))
446         if end is None:
447             end = _parse_secs(row.get("end_mmss"))
448         if start is None or end is None:
449             continue
450         records.append(RallyRecord(
451             rally_number=rn, start_time=start, end_time=end,
452             shots_count=safe_int(row.get("shots_count"), None),
453             ending_reason=(row.get("ending_reason") or "").strip() or None,
454             last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
455             score_before=(row.get("score_before") or "").strip() or None,
456             score_after=(row.get("score_after") or "").strip() or None,
457             notes=(row.get("notes") or "").strip() or None,
458             frame_min=-1, frame_max=-1, n_boxes=0,
459         ))
460     return records
461
462
463 def build_summary(records: List[RallyRecord], issues: List[Issue],
464                  source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
465     """Machine-readable validation summary (JSON / CI)."""
466     by_sev = {s: [i for i in issues if i.severity == s]
467              for s in ("blocker", "high", "medium", "low")}
468     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
469     return {
470         "source": source,
471         "fps": fps,
472         "n_rallies": len(records),
473         "n_issues": len(issues),
474         "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
475         "n_blockers": len(by_sev["blocker"]),
476         "passed": len(by_sev["blocker"]) == 0,
477         "flagged_rallies": flagged,
478         "issues": [{"severity": i.severity, "code": i.code,
479                    "rally_number": i.rally_number, "message": i.message} for i in
480 issues],
481     }
482
483 def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
-> str:
484     """Human validation report: scoreboard + issues by severity + rallies to review."""
485     n = len(records)
486     by_sev = {s: [i for i in issues if i.severity == s]
487              for s in ("blocker", "high", "medium", "low")}
488     flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
489     clean = n - len(flagged)
490     lines = [
491         f"# Delivery validation ? {source}",
492         "",
493         f"- rallies: **{n}**",
494         f"- issues: **{len(issues)}** (blocker {len(by_sev['blocker'])}, "
495         f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
496         {len(by_sev['low'])})",
497         f"- structurally-clean rallies: **{clean}/{n}**",
498         + (f" ({100 * clean // n}%" if n else ""),
499         f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
500         "",
501     ]
502     if issues:
503         for sev in ("blocker", "high", "medium", "low"):
504             grp = by_sev[sev]
505             if not grp:
506                 continue
507             lines.append(f"## {sev.upper()}")

```

```

507         lines += [f"- ({i.code}) {i.message}" for i in grp]
508         lines.append("")
509     else:
510         lines.append("No issues detected. ?\n")
511     if flagged:
512         lines.append(f"## Rallies to review: {flagged}")
513     return "\n".join(lines)
514
515
516 def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
517     """Auto-generate the annotatable manual-verification sheet (blank `true_` cols)."""
518     import csv
519     flags: Dict[int, List[str]] = {}
520     for i in issues:
521         if i.rally_number is not None:
522             flags.setdefault(i.rally_number, []).append(i.code)
523     os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
524     cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
525            "ending_reason", "last_shot_outcome", "auto_flags",
526            "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
527            "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
528     with open(out_path, "w", newline="", encoding="utf-8") as f:
529         w = csv.writer(f)
530         w.writerow(cols)
531         for r in sorted(records, key=lambda x: x.rally_number):
532             w.writerow([
533                 r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
534                 f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
535                 r.ending_reason or "", r.last_shot_outcome or "",
536                 ";".join(flags.get(r.rally_number, [])),
537                 "", "", "", "", "", "",
538             ])
539

```

15. user (2026-06-08T08:39:13.743Z)

```

1     import os
2     import sys
3     import time
4     import json
5     import csv
6     import argparse
7     from typing import List, Dict, Any, Tuple
8     from tabulate import tabulate
9
10    from src.core.config import load_config
11    from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12    from src.core.db import SportsDatabase
13    from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14    from src.core.schemas.badminton import BadmintonRally
15    from src.core.schemas.tennis import TennisPoint
16    from src.core.schemas.cricket import CricketDelivery
17    from src.core.schemas.pickleball import PickleballRally
18    from src.core.schemas.tabletennis import TableTennisRally
19    from src.core.validator import DatasetValidator
20    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23    class SportsDataCLI:
24        def __init__(self, config_path: str = "config/config.yaml"):
25            self.config_path = config_path
26            self.config = self._load_config()
27

```

```

28         # Resolve DB path
29         db_path = self.config.get("database_path", "data/sports_metadata.db")
30         self.db = SportsDatabase(db_path)
31         self.validator = DatasetValidator(config_path)
32
33     def _load_config(self) -> Dict[str, Any]:
34         if not os.path.exists(self.config_path):
35             print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36         return load_config(self.config_path)
37
38     def status(self):
39         """Displays status summary of all videos in the database."""
40         # Query total count by status via the public DB read API.
41         rows = self.db.get_status_counts() # list of (sport, status, cnt)
42
43         if not rows:
44             print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45             return
46
47         data = [[sport, status, cnt] for sport, status, cnt in rows]
48         print("\n=== Dataset Ingestion Status ===")
49         print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50         print()
51
52     def import_local(self, args):
53         """Manually registers a local video and annotation file into the database."""
54         sport_str = args.sport.lower()
55         if sport_str not in self.config.get("supported_sports", ["badminton"]):
56             print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57             sys.exit(1)
58
59         # Validate video path
60         if not os.path.exists(args.video_path):
61             print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63         # Validate annotation file
64         if not os.path.exists(args.annotations_path):
65             print(f"Error: Annotations file '{args.annotations_path}' not found.")
66             sys.exit(1)
67
68         match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69         video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71         # Parse license
72         try:
73             license_type = LicenseType(args.license)
74         except ValueError:
75             print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76             license_type = LicenseType.UNKNOWN
77
78         is_commercial = self.validator.check_license_commercial(license_type)
79
80         # 1. Parse Annotations
81         annotations = []
82         try:
83             annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84         except Exception as e:
85             print(f"Error parsing annotations: {e}")
86             sys.exit(1)
87

```



```

88         # 2. Run Validation
89         is_valid = True
90         errors = []
91         if sport_str == "badminton":
92             is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93         elif sport_str == "tennis":
94             is_valid, errors = self.validator.validate_tennis_points(annotations)
95         elif sport_str == "cricket":
96             is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97         elif sport_str == "pickleball":
98             is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99         elif sport_str == "tabletennis":
100             is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102         if not is_valid:
103             print(f"Error: Annotations failed validation!")
104             for err in errors:
105                 print(f" - {err}")
106             sys.exit(1)
107
108         # 3. Create Video Metadata
109         video_meta = VideoMetadata(
110             video_id=video_id,
111             sport=SportType(sport_str),
112             video_url=None,
113             local_path=args.video_path,
114             license_type=license_type,
115             license_url=None,
116             is_commercial=is_commercial,
117             status=CurationStatus.CURATED, # Custom local imports bypass staging by
default
118             match_name=match_name,
119             fps=args.fps,
120             resolution=args.resolution
121         )
122
123         # 4. Insert into Database
124         self.db.insert_video(video_meta)
125
126         count = 0
127         if sport_str == "badminton":
128             count = self.db.insert_badminton_rallies(annotations)
129         elif sport_str == "tennis":
130             count = self.db.insert_tennis_points(annotations)
131         elif sport_str == "cricket":
132             count = self.db.insert_cricket_deliveries(annotations)
133         elif sport_str == "pickleball":
134             count = self.db.insert_pickleball_rallies(annotations)
135         elif sport_str == "tabletennis":
136             count = self.db.insert_tabletennis_rallies(annotations)
137
138         print(f"\nSuccessfully imported local video '{match_name}':")
139         print(f" - Video ID: {video_id}")
140         print(f" - Sport: {sport_str}")
141         print(f" - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142         print(f" - Registered {count} annotated segments.\n")
143
144         # Per-video observability report (next to the video). Never raises.
145         from src.reporting import emit_import_report
146         src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147         report_paths = emit_import_report(
148             video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149             errors=[], count=count, sport=sport_str, source_format=src_fmt,

```

```

150         fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151     )
152     if report_paths:
153         print(f" - Observability report: {report_paths.get('technical_md')}")
154         print(f" - Customer summary      : {report_paths.get('customer_md')}\n")
155
156     def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157         """Parses simple JSON or CSV annotations files."""
158         parsed_items = []
159         ext = os.path.splitext(filepath)[1].lower()
160
161         if ext == '.json':
162             with open(filepath, 'r', encoding='utf-8') as f:
163                 data = json.load(f)
164
165             if not isinstance(data, list):
166                 raise ValueError("JSON file must contain a list of objects.")
167
168             # Uses the same safe casts as the CSV path: a missing/blank required
169             # time raises a clear ValueError (caught by the caller) instead of a
170             # bare KeyError, and optional ints degrade to sensible defaults.
171             for idx, obj in enumerate(data):
172                 if sport == "badminton":
173                     parsed_items.append(BadmintonRally(
174                         video_id=video_id,
175                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                         start_time=_safe_float(obj.get("start_time")),
177                         end_time=_safe_float(obj.get("end_time")),
178                         score_before=obj.get("score_before"),
179                         score_after=obj.get("score_after"),
180                         shots_count=_safe_int(obj.get("shots_count"), None),
181                         ending_reason=obj.get("ending_reason"),
182                         last_shot_outcome=obj.get("last_shot_outcome")
183                     ))
184                 elif sport == "tennis":
185                     parsed_items.append(TennisPoint(
186                         video_id=video_id,
187                         point_number=_safe_int(obj.get("point_number"), idx + 1),
188                         start_time=_safe_float(obj.get("start_time")),
189                         end_time=_safe_float(obj.get("end_time")),
190                         set_score=obj.get("set_score"),
191                         game_score=obj.get("game_score"),
192                         server=obj.get("server"),
193                         ending_type=obj.get("ending_type")
194                     ))
195                 elif sport == "cricket":
196                     parsed_items.append(CricketDelivery(
197                         video_id=video_id,
198                         over=_safe_int(obj.get("over"), 0),
199                         ball=_safe_int(obj.get("ball"), 0),
200                         start_time=_safe_float(obj.get("start_time")),
201                         end_time=_safe_float(obj.get("end_time")),
202                         batsman=obj.get("batsman"),
203                         bowler=obj.get("bowler"),
204                         runs=_safe_int(obj.get("runs"), None),
205                         wicket=bool(obj.get("wicket", False)),
206                         extras=_safe_int(obj.get("extras"), 0)
207                     ))
208                 elif sport == "pickleball":
209                     parsed_items.append(PickleballRally(
210                         video_id=video_id,
211                         rally_number=_safe_int(obj.get("rally_number"), idx + 1),
212                         start_time=_safe_float(obj.get("start_time")),
213                         end_time=_safe_float(obj.get("end_time")),

```

```

214         score_before=obj.get("score_before"),
215         score_after=obj.get("score_after"),
216         shots_count=_safe_int(obj.get("shots_count"), None),
217         ending_reason=obj.get("ending_reason"),
218         last_shot_outcome=obj.get("last_shot_outcome")
219     ))
220     elif sport == "tabletennis":
221         parsed_items.append(TableTennisRally(
222             video_id=video_id,
223             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
224             start_time=_safe_float(obj.get("start_time")),
225             end_time=_safe_float(obj.get("end_time")),
226             score_before=obj.get("score_before"),
227             score_after=obj.get("score_after"),
228             shots_count=_safe_int(obj.get("shots_count"), None),
229             ending_reason=obj.get("ending_reason"),
230             last_shot_outcome=obj.get("last_shot_outcome")
231         ))
232     elif ext == '.csv':
233         with open(filepath, 'r', encoding='utf-8') as f:
234             reader = csv.DictReader(f)
235             # Normalize headers
236             reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
237
238         for idx, row in enumerate(reader):
239             if sport == "badminton":
240                 parsed_items.append(BadmintonRally(
241                     video_id=video_id,
242                     rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                     start_time=_safe_float(row.get("start_time")),
244                     end_time=_safe_float(row.get("end_time")),
245                     score_before=row.get("score_before"),
246                     score_after=row.get("score_after"),
247                     shots_count=_safe_int(row.get("shots_count"), None),
248                     ending_reason=row.get("ending_reason"),
249                     last_shot_outcome=row.get("last_shot_outcome")
250                 ))
251             elif sport == "tennis":
252                 parsed_items.append(TennisPoint(
253                     video_id=video_id,
254                     point_number=_safe_int(row.get("point_number"), idx + 1),
255                     start_time=_safe_float(row.get("start_time")),
256                     end_time=_safe_float(row.get("end_time")),
257                     set_score=row.get("set_score"),
258                     game_score=row.get("game_score"),
259                     server=row.get("server"),
260                     ending_type=row.get("ending_type")
261                 ))
262             elif sport == "cricket":
263                 parsed_items.append(CricketDelivery(
264                     video_id=video_id,
265                     over=_safe_int(row.get("over"), 0),
266                     ball=_safe_int(row.get("ball"), 0),
267                     start_time=_safe_float(row.get("start_time")),
268                     end_time=_safe_float(row.get("end_time")),
269                     batsman=row.get("batsman"),
270                     bowler=row.get("bowler"),
271                     runs=_safe_int(row.get("runs"), None),
272                     wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                     extras=_safe_int(row.get("extras"), 0)
274                 ))
275             elif sport == "pickleball":
276                 parsed_items.append(PickleballRally(

```

```

277         video_id=video_id,
278         rally_number=_safe_int(row.get("rally_number"), idx + 1),
279         start_time=_safe_float(row.get("start_time")),
280         end_time=_safe_float(row.get("end_time")),
281         score_before=row.get("score_before"),
282         score_after=row.get("score_after"),
283         shots_count=_safe_int(row.get("shots_count"), None),
284         ending_reason=row.get("ending_reason"),
285         last_shot_outcome=row.get("last_shot_outcome")
286     ))
287     elif sport == "tabletennis":
288         parsed_items.append(TableTennisRally(
289             video_id=video_id,
290             rally_number=_safe_int(row.get("rally_number"), idx + 1),
291             start_time=_safe_float(row.get("start_time")),
292             end_time=_safe_float(row.get("end_time")),
293             score_before=row.get("score_before"),
294             score_after=row.get("score_after"),
295             shots_count=_safe_int(row.get("shots_count"), None),
296             ending_reason=row.get("ending_reason"),
297             last_shot_outcome=row.get("last_shot_outcome")
298         ))
299     else:
300         raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")
301
302     return parsed_items
303
304 def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
305     """Download one video at best resolution, retrying transient failures.
306
307     A below-min-height result almost always means YouTube rate-limited the
308     request and served a low fallback (not that the video lacks HD), so we
309     retry with a growing backoff. Returns (DownloadResult, None) on success,
310     or (None, failure_reason) once retries are exhausted. A below-min file is
311     discarded each attempt so it never lands in the DB.
312     """
313     attempts = max(1, args.retries + 1)
314     reason = "unknown error"
315     for i in range(attempts):
316         result = download_best_video(
317             url, dest_path,
318             max_height=max_height, max_size_mb=max_size_mb,
319             cookies_from_browser=args.cookies_from_browser,
320             cookies_file=args.cookies,
321             player_client=args.player_client,
322         )
323         if result.success and (result.height is None or result.height >=
args.min_height):
324             return result, None
325
326         if not result.success:
327             reason = result.error or "unknown error"
328         else:
329             reason = (f"got {result.resolution_label} (< min-height
{args.min_height})p) ? "
330                     f"YouTube served a low format (rate-limited?)")
331         try:
332             os.remove(result.path)
333         except OSError:
334             pass
335
336     if i < attempts - 1:
337         backoff = args.retry_backoff * (i + 1)
338         print(f" attempt {i+1}/{attempts} failed: {reason}\n retrying in

```

```

{backoff:.0f}s...)
339         time.sleep(backoff)
340     return None, reason
341
342     def fetch(self, args):
343         """(Re)download videos at best available resolution, updating the DB in place.
344
345         This is the in-place upgrade path: rows, rally annotations, and curation
346         status are all preserved ? only the video file plus its `local_path`,
347         `resolution`, and `fps` are refreshed. By default it skips videos that
348         already have a local file at or above `--min-height`, so re-runs are cheap
349         and resumable; `--force` re-fetches everything.
350         """
351         storage = self.config.get("video_storage", {})
352         local_dir = storage.get("local_dir", "./data/videos")
353         max_download_size_mb = storage.get("max_download_size_mb")
354         max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
355         os.makedirs(local_dir, exist_ok=True)
356
357         commercial_only = not args.include_noncommercial
358         if args.status == "all":
359             statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
360         else:
361             statuses = [CurationStatus(args.status)]
362
363         supported = self.config.get("supported_sports", ["badminton"])
364         sport_strs = [args.sport] if args.sport else supported
365
366         # Collect target videos.
367         videos = []
368         for sport_str in sport_strs:
369             try:
370                 sport = SportType(sport_str.lower())
371             except ValueError:
372                 print(f"Warning: skipping unknown sport '{sport_str}'.")
373                 continue
374             for status in statuses:
375                 videos.extend(self.db.get_videos_by_status(status, sport))
376         if args.video_id:
377             videos = [v for v in videos if v.video_id == args.video_id]
378
379         if not videos:
380             print("\nNo matching videos to fetch.\n")
381             return
382
383         results = [] # (video_id, action, detail)
384         for video in videos:
385             if commercial_only and not video.is_commercial:
386                 results.append((video.video_id, "skip", "non-commercial"))
387                 continue
388             if not video.video_url or "watch?v=example" in video.video_url:
389                 # Either no URL, or the parser's placeholder (the ShuttleSet catalog
390                 # had no URL for this match) ? nothing to download.
391                 results.append((video.video_id, "skip", "no real source URL"))
392                 continue
393
394             dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")
395
396             # Skip if we already have a good-enough local file (unless --force).
397             if not args.force and os.path.exists(dest_path):
398                 _, height, _ = probe_resolution_fps(dest_path)
399                 if height is not None and height >= args.min_height:
400                     results.append((video.video_id, "have", f"{height_to_label(height)}
already"))

```

```

401         continue
402
403     print(f"Fetching {video.video_id} ({video.match_name})...")
404     result, fail_reason = self._fetch_one_with_retry(
405         video.video_url, dest_path, args, max_height, max_download_size_mb)
406     if fail_reason is not None:
407         results.append((video.video_id, "FAIL", fail_reason))
408         if args.sleep:
409             time.sleep(args.sleep)
410         continue
411
412     # Update only the file-derived fields; preserve everything else (UPSERT
413     # mutates the row in place, leaving rallies intact).
414     video.local_path = result.path
415     if result.resolution_label:
416         video.resolution = result.resolution_label
417     if result.fps:
418         video.fps = result.fps
419     self.db.insert_video(video)
420
421     detail = (f"{result.resolution_label or '?}'"
422             f"{f' {result.fps:g}fps' if result.fps else ''},
423 {result.size_mb:.0f} MB")
424     results.append((video.video_id, "fetched", detail))
425     if args.sleep:
426         time.sleep(args.sleep)
427
428     print("\n=== Fetch summary ===")
429     print(tabulate([[vid, action, detail] for vid, action, detail in results],
430                  headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
431     fetched = sum(1 for _, a, _ in results if a == "fetched")
432     failed = [r for r in results if r[1] == "FAIL"]
433     print(f"\nFetched/updated : {fetched}    Already-good : {sum(1 for _, a, _ in
434 results if a=='have'))}    "
435           f"Skipped : {sum(1 for _, a, _ in results if a=='skip')}}    Failed :
436 {len(failed)}}")
437     if failed:
438         print("Failures (left unchanged in DB):")
439         for vid, _, detail in failed:
440             print(f" - {vid}: {detail}")
441         print()
442
443 def export_eval_set(self, args):
444     """Export curated annotations as indexer-ready eval/training sets.
445
446     Emits, per qualifying video, a `//.csv` in the
447     indexer's `start,end` ground-truth format (decimal seconds), plus a single
448     `/manifest.json` that pairs each CSV with its video file. The
449     manifest is the contract the downstream eval/training driver reads: the
450     indexer keys videos by `file MD5`, not by our `video_id`, so it must run
451     against the exact `local_path`/`video_url` recorded here.
452
453     By default only CURATED + commercially-safe videos are exported, mirroring
454     the license gate already enforced elsewhere in the toolchain.
455     """
456     out_dir = args.out_dir
457     commercial_only = not args.include_noncommercial
458     statuses = [CurationStatus.CURATED]
459     if args.include_staging:
460         statuses.append(CurationStatus.STAGING)
461
462     supported = self.config.get("supported_sports", ["badminton"])
463     sport_strs = [args.sport] if args.sport else supported
464
465     manifest = []

```

```

463         skipped_noncomm = 0
464         skipped_empty = 0
465
466     for sport_str in sport_strs:
467         try:
468             sport = SportType(sport_str.lower())
469         except ValueError:
470             print(f"Warning: skipping unknown sport '{sport_str}'.")
471             continue
472
473         videos = []
474         for status in statuses:
475             videos.extend(self.db.get_videos_by_status(status, sport))
476
477         for video in videos:
478             if commercial_only and not video.is_commercial:
479                 skipped_noncomm += 1
480                 continue
481
482             intervals = self.db.get_segment_intervals(video.video_id, sport)
483             if not intervals:
484                 skipped_empty += 1
485                 continue
486
487             sport_dir = os.path.join(out_dir, sport.value)
488             os.makedirs(sport_dir, exist_ok=True)
489             csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
490             with open(csv_path, "w", newline="", encoding="utf-8") as f:
491                 writer = csv.writer(f)
492                 writer.writerow(["start", "end"])
493                 for start, end in intervals:
494                     writer.writerow([start, end])
495
496             manifest.append({
497                 "video_id": video.video_id,
498                 "sport": sport.value,
499                 "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
500                 "local_path": video.local_path,
501                 "video_url": video.video_url,
502                 "match_name": video.match_name,
503                 "fps": video.fps,
504                 "resolution": video.resolution,
505                 "license_type": video.license_type.value,
506                 "is_commercial": video.is_commercial,
507                 "status": video.status.value,
508                 "num_segments": len(intervals),
509             })
510
511     os.makedirs(out_dir, exist_ok=True)
512     manifest_doc = {
513         "commercial_only": commercial_only,
514         "statuses": [s.value for s in statuses],
515         "total_videos": len(manifest),
516         "total_segments": sum(m["num_segments"] for m in manifest),
517         "eval_sets": manifest,
518     }
519     manifest_path = os.path.join(out_dir, "manifest.json")
520     with open(manifest_path, "w", encoding="utf-8") as f:
521         json.dump(manifest_doc, f, indent=2)
522
523     print(f"\n=== Exported eval/training set to '{out_dir}' ===")
524     if manifest:
525         summary = [[m["sport"], m["video_id"], m["num_segments"],
526                     "YES" if m["is_commercial"] else "NO",
527                     os.path.basename(m["local_path"]) if m["local_path"] else "(no

```

```

local file)"]
528             for m in manifest]
529             print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
530             print(f"\nVideos exported : {len(manifest)}")
531             print(f"Total segments : {manifest_doc['total_segments']}")
532             if skipped_noncomm:
533                 print(f"Skipped (non-commercial) : {skipped_noncomm} (pass --include-
noncommercial to include)")
534             if skipped_empty:
535                 print(f"Skipped (no annotations) : {skipped_empty}")
536             print(f"Manifest : {manifest_path}\n")
537
538     def convert_cvat(self, args):
539         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
540
541         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
542         attributes) on the ingestion side: each rally's time is recomputed from
543         its frame range at the video's true fps, and content defects (overlaps,
544         zero-duration, off-vocabulary endings, coarse sampling) are written to an
545         issues report rather than silently "fixed". The emitted CSV imports via
546         `import-local`; any blocker issue must be resolved first, since the
547         importer's validator rejects overlapping / zero-duration rallies.
548         """
549         from src.parsers.cvat.rally_cvat import (
550             convert, write_canonical_csv, render_issue_report,
551         )
552         try:
553             result = convert(args.input, fps=args.fps,
554                             video_path=args.video_path, step=args.step)
555         except (FileNotFoundError, ValueError) as e:
556             print(f"Error: {e}")
557             sys.exit(1)
558
559         write_canonical_csv(result.records, args.out)
560
561         # Observability report next to the canonical CSV. Never raises.
562         from src.reporting import emit_delivery_report
563         src_name = os.path.splitext(os.path.basename(args.input))[0]
564         dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
565         emit_delivery_report(
566             out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
567             records=result.records, issues=result.issues, fps=result.fps,
568             video_path=args.video_path,
569             video_duration_s=dur, source_format="cvat", step=result.step,
570         )
571         report = render_issue_report(result, os.path.basename(args.input))
572         if args.issues:
573             with open(args.issues, "w", encoding="utf-8") as f:
574                 f.write(report)
575         print(report)
576         print(f"Canonical CSV written: {args.out} ({len(result.records)} rallies)")
577         if result.n_blockers:
578             print(f"\n {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
579                 f"this file until they are resolved (by the vendor or manual
review).")
580         else:
581             print("\n No blockers. Next:")
582             print(f"    python -m src.cli.curate import-local --sport <sport> "
583                 f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
584

```



```

585 def validate_delivery(self, args):
586     """Run the full validation suite on a rally delivery and emit a report.
587
588     Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
589     CSV (--csv). Writes, into --out-dir:
590         <id>_report.md    human report (scoreboard + issues + rallies to review)
591         <id>_report.json  machine summary (for CI / a bulk-ingest loop)
592         <id>_review.csv   the annotatable manual-verification sheet
593     Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
594     """
595     import json
596     from src.parsers.cvat.rally_cvat import (
597         convert, load_canonical_csv, detect_issues,
598         build_summary, render_validation_md, write_review_csv,
599     )
600     vid = args.video_id or "delivery"
601     fps = None
602     step = None
603     stop_frame = None
604     if args.input:
605         try:
606             result = convert(args.input, fps=args.fps, video_path=args.video_path)
607         except (FileNotFoundError, ValueError) as e:
608             print(f"Error: {e}")
609             sys.exit(1)
610         records, issues, fps = result.records, result.issues, result.fps
611         step = result.step
612         stop_frame = result.meta.get("stop_frame")
613     elif args.csv:
614         try:
615             records = load_canonical_csv(args.csv)
616         except FileNotFoundError as e:
617             print(f"Error: {e}")
618             sys.exit(1)
619         if not records:
620             print(f"Error: no usable rally rows in {args.csv}")
621             sys.exit(1)
622         issues = detect_issues(records)
623     else:
624         print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
625         sys.exit(1)
626
627     os.makedirs(args.out_dir, exist_ok=True)
628     summary = build_summary(records, issues, source=vid, fps=fps)
629     md = render_validation_md(records, issues, source=vid)
630     json_path = os.path.join(args.out_dir, f"{vid}_report.json")
631     md_path = os.path.join(args.out_dir, f"{vid}_report.md")
632     review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
633     with open(json_path, "w", encoding="utf-8") as f:
634         json.dump(summary, f, indent=2)
635     with open(md_path, "w", encoding="utf-8") as f:
636         f.write(md)
637     write_review_csv(records, issues, review_path)
638
639     # Observability report (envelope JSON + technical md + customer summary),
640     # colocated with the existing artifacts. Never raises.
641     from src.reporting import emit_delivery_report
642     duration_s = (stop_frame / fps) if (stop_frame and fps) else None
643     report_paths = emit_delivery_report(
644         out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
645         video_path=args.video_path, video_duration_s=duration_s,
646         source_format=("cvat" if args.input else "csv"), step=step,
647     )
648
649     print(md)

```

```

650         print(f"\nwrote:\n {md_path}\n {json_path}\n {review_path}")
651         if report_paths:
652             print(f" {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n {report_paths.get('json')}")
653         if summary["n_blockers"]:
654             print(f"\n GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
655             sys.exit(2)
656         print("\n GATE: PASS ? review the flagged rallies, then `import-local`.")
657
658     def curate(self):
659         """Interactive loop to review and curate staging video annotations."""
660         staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
661         if not staging_videos:
662             print("\nNo staging video annotations to review.\n")
663             return
664
665         print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
666         table_data = []
667         for idx, video in enumerate(staging_videos):
668             table_data.append([
669                 idx + 1,
670                 video.video_id,
671                 video.match_name,
672                 video.sport.value,
673                 video.license_type.value,
674                 "YES" if video.is_commercial else "NO"
675             ])
676
677         print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
678         print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))
679
680         choice = input("> ").strip()
681         if choice.lower() == 'q':
682             return
683
684         try:
685             idx = int(choice) - 1
686             if idx < 0 or idx >= len(staging_videos):
687                 print("Invalid choice.")
688                 return
689             self._curate_video_interactive(staging_videos[idx])
690         except ValueError:
691             print("Invalid input.")
692
693     def _curate_video_interactive(self, video: VideoMetadata):
694         """Displays details of a specific staging video and prompts for curation
action."""
695
696         # Load and count events
697         rallies = []
698         pts = []
699         delivs = []
700         if video.sport == SportType.BADMINTON:
701             rallies = self.db.get_badminton_rallies(video.video_id)
702         elif video.sport == SportType.PICKLEBALL:
703             rallies = self.db.get_pickleball_rallies(video.video_id)
704         elif video.sport == SportType.TABLE_TENNIS:
705             rallies = self.db.get_tabletennis_rallies(video.video_id)
706         elif video.sport == SportType.TENNIS:
707             pts = self.db.get_tennis_points(video.video_id)
708         elif video.sport == SportType.CRICKET:
709             delivs = self.db.get_cricket_deliveries(video.video_id)

```

```

710
711         while True:
712             print("\n===== REVIEWING VIDEO =====")
713             print(f"Match Name      : {video.match_name}")
714             print(f"Video ID       : {video.video_id}")
715             print(f"Sport          : {video.sport.value}")
716             print(f"Video URL      : {video.video_url or 'N/A'}")
717             print(f"Local Path     : {video.local_path or 'N/A'}")
718             print(f"License        : {video.license_type.value} ({video.license_url or 'no
URL'})")
719             print(f"Commercial OK: {video.is_commercial}")
720
721             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
722                 print(f"Total Rallies: {len(rallies)}")
723                 if rallies:
724                     print("Sample rallies:")
725                     sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
726                     print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
727                 elif video.sport == SportType.TENNIS:
728                     print(f"Total Points : {len(pts)}")
729                     if pts:
730                         print("Sample points:")
731                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732                         print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733                 elif video.sport == SportType.CRICKET:
734                     print(f"Total Balls  : {len(delivs)}")
735                     if delivs:
736                         print("Sample deliveries:")
737                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740             print("\nChoose Curation Action:")
741             print("  [a] Approve (promote to CURATED)")
742             print("  [r] Reject (delete from database)")
743             print("  [p] Play a rally/point segment in browser")
744             print("  [k] Keep in Staging (do nothing and go back)")
745
746             action = input("> ").strip().lower()
747             if action == 'a':
748                 self.db.update_status(video.video_id, CurationStatus.CURATED)
749                 print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750                 break
751             elif action == 'r':
752                 self.db.delete_video(video.video_id)
753                 print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754                 break
755             elif action == 'k':
756                 print("Curation state unchanged.\n")
757                 break
758             elif action == 'p':
759                 self._play_segment_interactive(video, rallies, pts, delivs)
760             else:
761                 print("Invalid action selected.")
762
763         def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764             import webbrowser
765

```

```

766         # Get start/end based on user input
767         start_time = None
768         end_time = None
769         label = "segment"
770
771         if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
772             if not rallies:
773                 print("No rallies to play.")
774                 return
775             idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
776             try:
777                 rally_num = int(idx_str)
778                 rally = next((r for r in rallies if r.rally_number == rally_num), None)
779                 if rally:
780                     start_time = rally.start_time
781                     end_time = rally.end_time
782                     label = f"Rally {rally_num}"
783             except ValueError:
784                 pass
785         elif video.sport == SportType.TENNIS:
786             if not pts:
787                 print("No points to play.")
788                 return
789             idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
790             try:
791                 pt_num = int(idx_str)
792                 pt = next((p for p in pts if p.point_number == pt_num), None)
793                 if pt:
794                     start_time = pt.start_time
795                     end_time = pt.end_time
796                     label = f"Point {pt_num}"
797             except ValueError:
798                 pass
799         elif video.sport == SportType.CRICKET:
800             if not delivs:
801                 print("No deliveries to play.")
802                 return
803             over_str = input("Enter Over number: ").strip()
804             ball_str = input("Enter Ball number: ").strip()
805             try:
806                 o_num = int(over_str)
807                 b_num = int(ball_str)
808                 d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
809             if d:
810                 start_time = d.start_time
811                 end_time = d.end_time
812                 label = f"Over {o_num} Ball {b_num}"
813             except ValueError:
814                 pass
815
816         if start_time is None or end_time is None:
817             print("Segment not found or invalid input.")
818             return
819
820         print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")
821
822         # Check video location
823         play_url = None
824         if video.local_path and os.path.exists(video.local_path):
825             abs_path = os.path.abspath(video.local_path)
826             # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
827             play_url = f"file:/// {abs_path.replace(os.sep,

```

```

'/'}}#t={start_time},{end_time}"
828         elif video.video_url:
829             # YouTube URL: seek to start time
830             if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
831                 t_sec = int(start_time)
832                 sep = "&" if "?" in video.video_url else "?"
833                 play_url = f"{video.video_url}{sep}t={t_sec}s"
834             else:
835                 play_url = video.video_url
836
837         if play_url:
838             print(f"Opening browser: {play_url}")
839             webbrowser.open(play_url)
840         else:
841             print("Error: No local video file or remote URL available to play.")
842
843
844     def main():
845         parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
846         subparsers = parser.add_subparsers(dest="command")
847
848         # Status command
849         subparsers.add_parser("status", help="Display summary status of sports metadata
database")
850
851         # Curate command
852         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
853
854         # Import-local command
855         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
856         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
857         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871             help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873             help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875             help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",

```

```

877                                     help="Re-download even if a good-enough local file already
exists")
878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879                                     help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
881                                     help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                     "A Premium login gives reliable best-resolution
downloads. "
883                                     "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884         fetch_parser.add_argument("--cookies", metavar="FILE",
885                                     help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
886         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                                     help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                                     "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                                     help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
891         fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
892                                     help="Retries per video when a download fails or returns a
low format (default: 2)")
893         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
894                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
895
896         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
897         cvat_parser = subparsers.add_parser(
898             "convert-cvat",
899             help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
900         cvat_parser.add_argument("--input", required=True,
901                                     help="Path to the CVAT XML export (image- or track-mode)")
902         cvat_parser.add_argument("--out", required=True,
903                                     help="Path to write the canonical per-rally CSV (import-
local input)")
904         cvat_parser.add_argument("--video-path",
905                                     help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
906         cvat_parser.add_argument("--fps", type=float,
907                                     help="Override fps instead of probing the video")
908         cvat_parser.add_argument("--step", type=int,
909                                     help="Frame sampling step (default: read from CVAT meta,
else inferred)")
910         cvat_parser.add_argument("--issues",
911                                     help="Path to also write the human-readable issues report")
912
913         # Validate-delivery command ? run the full check suite + emit a structured report
914         vd_parser = subparsers.add_parser(
915             "validate-delivery",
916             help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
917         vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
918         vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
919         vd_parser.add_argument("--video-path",
920                                     help="Source video; fps via ffprobe (when using --input)")

```

```

921     vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
922     vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
923     vd_parser.add_argument("--out-dir", default="data/validation",
924                             help="Where to write the report + review sheet")
925
926     # Export command
927     export_parser = subparsers.add_parser(
928         "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
929     export_parser.add_argument("--out-dir", default="data/eval_export",
930                                 help="Output directory for per-video CSVs and
manifest.json")
931     export_parser.add_argument("--sport",
932                                 help="Restrict to a single sport (default: all
supported_sports)")
933     export_parser.add_argument("--include-staging", action="store_true",
934                                 help="Also export videos still in staging (default:
curated only)")
935     export_parser.add_argument("--include-noncommercial", action="store_true",
936                                 help="Include videos whose license is not commercially
safe (default: commercial only)")
937
938     args = parser.parse_args()
939     cli = SportsDataCLI()
940
941     if args.command == "status":
942         cli.status()
943     elif args.command == "curate":
944         cli.curate()
945     elif args.command == "import-local":
946         cli.import_local(args)
947     elif args.command == "convert-cvat":
948         cli.convert_cvat(args)
949     elif args.command == "validate-delivery":
950         cli.validate_delivery(args)
951     elif args.command == "export":
952         cli.export_eval_set(args)
953     elif args.command == "fetch":
954         cli.fetch(args)
955     else:
956         # Default: print help
957         parser.print_help()
958
959     if __name__ == "__main__":
960         main()
961

```

16. assistant (2026-06-08T08:39:16.461Z)

Perfect. Now let me check for any additional schema files to understand the full picture:

17. user (2026-06-08T08:39:16.896Z)

```

1     from pydantic import BaseModel, Field, model_validator
2     from typing import Optional
3
4     class PickleballRally(BaseModel):
5         video_id: str
6         rally_number: int = Field(..., description="Chronological rally index in the match")
7         start_time: float = Field(..., description="Start of rally in seconds (serve
contact)")
8         end_time: float = Field(..., description="End of rally in seconds (ball dead)")
9         score_before: Optional[str] = Field(None, description="Score before the rally (e.g.

```

```

'0-0')")
10         score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11         shots_count: Optional[int] = Field(None, description="Number of paddle-ball contacts
in the rally (serve = shot 1; bounces are not shots)")
12         ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13         last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15         @model_validator(mode='after')
16         def validate_timestamps(self) -> 'PickleballRally':
17             if self.start_time < 0:
18                 raise ValueError("start_time must be non-negative")
19             if self.end_time <= self.start_time:
20                 raise ValueError("end_time must be strictly greater than start_time")
21             return self
22

```

18. user (2026-06-08T08:39:17.102Z)

```

1         from pydantic import BaseModel, Field, model_validator
2         from typing import Optional
3
4         class TableTennisRally(BaseModel):
5             video_id: str
6             rally_number: int = Field(..., description="Chronological rally index in the match")
7             start_time: float = Field(..., description="Start of rally in seconds (serve racket-
ball contact)")
8             end_time: float = Field(..., description="End of rally in seconds (ball dead)")
9             score_before: Optional[str] = Field(None, description="Score before the rally (e.g.
'0-0')")
10            score_after: Optional[str] = Field(None, description="Score after the rally (e.g.
'1-0')")
11            shots_count: Optional[int] = Field(None, description="Number of racket-ball contacts
in the rally (serve = shot 1; table bounces are not shots)")
12            ending_reason: Optional[str] = Field(None, description="Why the rally ended:
'winner' | 'forced_error' | 'unforced_error' | 'service_fault' | 'let' | 'other'")
13            last_shot_outcome: Optional[str] = Field(None, description="Where the last shot
landed: 'in' | 'net' | 'out' | 'n_a' (n_a for service_fault/let)")
14
15            @model_validator(mode='after')
16            def validate_timestamps(self) -> 'TableTennisRally':
17                if self.start_time < 0:
18                    raise ValueError("start_time must be non-negative")
19                if self.end_time <= self.start_time:
20                    raise ValueError("end_time must be strictly greater than start_time")
21                return self
22

```

19. assistant (2026-06-08T08:39:19.305Z)

Now let me check for other parsers to see how they interact with the canonical schema:

20. user (2026-06-08T08:39:19.686Z)

```

1         import csv
2         import os
3         from typing import Tuple, List, Dict, Any, Optional
4         from src.parsers.base_parser import BaseParser
5         from src.core.parsing import safe_int as _safe_int, parse_time_string
6         from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
7         from src.core.schemas.badminton import BadmintonRally
8         from src.core.validator import DatasetValidator

```



```

9
10
11 class CoachAIBadmintonParser(BaseParser):
12     def __init__(
13         self,
14         start_buffer: float = 1.0,
15         end_buffer: float = 2.0,
16         validator: Optional[DatasetValidator] = None,
17     ):
18         """
19         Args:
20             start_buffer: Seconds to subtract from first stroke (serve) time.
21             end_buffer: Seconds to add to last stroke time.
22             validator: Used to decide commercial-license safety from the config
23                 whitelist. Defaults to a DatasetValidator with the default config so
24                 license classification stays consistent across the codebase.
25         """
26         self.start_buffer = start_buffer
27         self.end_buffer = end_buffer
28         self.validator = validator or DatasetValidator()
29
30     def parse(self, filepath: str, **kwargs) -> Tuple[VideoMetadata,
List[BadmintonRally]]:
31         """
32         Parses a CoachAI/ShuttleSet CSV format.
33
34         Expected CSV Columns (any subset or mapped names):
35             - set: Set number (1, 2, 3)
36             - rally: Rally number
37             - ball_round: Shot number within the rally (1, 2, 3...)
38             - time: Hitting time of the shot (in seconds)
39             - type: Stroke type (e.g., smash, lob, net...)
40             - score_a / score_b: Current scores
41         """
42         if not os.path.exists(filepath):
43             raise FileNotFoundError(f"File not found: {filepath}")
44
45         # Extract video details from kwargs or name
46         match_name = kwargs.get("match_name") or
os.path.basename(filepath).replace(".csv", "")
47         video_id = kwargs.get("video_id") or match_name.lower().replace(" ", "_")
48         video_url = kwargs.get("video_url") or "https://youtube.com/watch?v=example"
49         license_str = kwargs.get("license") or "MIT"
50
51         # Check if license matches commercial whitelist
52         try:
53             license_type = LicenseType(license_str)
54         except ValueError:
55             license_type = LicenseType.UNKNOWN
56
57         # Group rows by set & rally
58         # Key: (set_num, rally_num) -> List of rows (dict)
59         rallies_data: Dict[Tuple[int, int], List[Dict[str, Any]]] = {}
60
61         with open(filepath, mode='r', encoding='utf-8') as f:
62             reader = csv.DictReader(f)
63             # Normalize column names to lowercase/strip
64             reader.fieldnames = [name.lower().strip() for name in reader.fieldnames] if
reader.fieldnames else []
65
66             for row in reader:
67                 # Resolve key columns (tolerate empty cells and float-strings)
68                 set_num = _safe_int(row.get('set'), 1)
69                 rally_num = _safe_int(row.get('rally'), 0)
70                 ball_round = _safe_int(row.get('ball_round', row.get('ball_seq')), 1)

```

```

71
72         # Resolve time (safely parse float or HH:MM:SS strings)
73         raw_time = row.get('time', row.get('timestamp', row.get('sec', '0.0')))
74         time_val = parse_time_string(raw_time)
75
76         key = (set_num, rally_num)
77         if key not in rallies_data:
78             rallies_data[key] = []
79
80         rallies_data[key].append({
81             'ball_round': ball_round,
82             'time': time_val,
83             'type': row.get('type', ''),
84             'score_a': row.get('score_a', row.get('round_score_a', '0')),
85             'score_b': row.get('score_b', row.get('round_score_b', '0'))
86         })
87
88     # First pass: build raw rally records (unbuffered stroke bounds) in chrono
order.
89     raw_rallies: List[Dict[str, Any]] = []
90     for key in sorted(rallies_data.keys()):
91         rows_sorted = sorted(rallies_data[key], key=lambda x: x['ball_round'])
92         if not rows_sorted:
93             continue
94         first_stroke = rows_sorted[0]
95         last_stroke = rows_sorted[-1]
96         raw_rallies.append({
97             'first_time': first_stroke['time'],
98             'last_time': last_stroke['time'],
99             'ending_reason': last_stroke['type'] if last_stroke['type'] else None,
100             'shots_count': len(rows_sorted),
101             'score_before': f"{first_stroke['score_a']}-{first_stroke['score_b']}",
102             'score_after': f"{last_stroke['score_a']}-{last_stroke['score_b']}",
103         })
104
105     # Second pass: apply start/end buffers, then resolve any boundary where the
106     # padding of two adjacent rallies would collide by meeting at the midpoint of
107     # the gap between their nearest strokes. This keeps the buffers from ever
108     # producing an artificial overlap (which the validator would otherwise reject).
109     starts = [max(0.0, rr['first_time'] - self.start_buffer) for rr in raw_rallies]
110     ends = [rr['last_time'] + self.end_buffer for rr in raw_rallies]
111     for i in range(len(raw_rallies) - 1):
112         if ends[i] > starts[i + 1]:
113             midpoint = (raw_rallies[i]['last_time'] + raw_rallies[i +
114 1]['first_time']) / 2
115             ends[i] = min(ends[i], midpoint)
116             starts[i + 1] = max(starts[i + 1], midpoint)
117
118     rallies: List[BadmintonRally] = []
119     for i, rr in enumerate(raw_rallies):
120         start_time = starts[i]
121         end_time = ends[i]
122         # Guarantee a strictly positive duration even after clamping.
123         if end_time <= start_time:
124             end_time = start_time + 0.001
125
126         rallies.append(BadmintonRally(
127             video_id=video_id,
128             rally_number=i + 1,
129             start_time=start_time,
130             end_time=end_time,
131             score_before=rr['score_before'],
132             score_after=rr['score_after'],
133             shots_count=rr['shots_count'],
134             ending_reason=rr['ending_reason']

```

```

134         ))
135
136     # Populate VideoMetadata
137     video_metadata = VideoMetadata(
138         video_id=video_id,
139         sport=SportType.BADMINTON,
140         video_url=video_url,
141         local_path=kwargs.get("local_path", None),
142         license_type=license_type,
143         license_url=kwargs.get("license_url", None),
144         is_commercial=self.validator.check_license_commercial(license_type),
145         status=CurationStatus.STAGING,
146         match_name=match_name,
147         fps=kwargs.get("fps", 30.0),
148         resolution=kwargs.get("resolution", "1080p")
149     )
150     return video_metadata, rallies
151

```

21. assistant (2026-06-08T08:39:21.822Z)

Now let me look at the database module to understand how these records are persisted:

22. user (2026-06-08T08:39:22.356Z)

567 C:\Users\avidu\Projects\sports-data-collector\src\core\db.py

23. user (2026-06-08T08:39:24.501Z)

```

1     import sqlite3
2     import os
3     from contextlib import contextmanager
4     from typing import List, Dict, Any, Optional
5     from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6     from src.core.schemas.badminton import BadmintonRally
7     from src.core.schemas.tennis import TennisPoint
8     from src.core.schemas.cricket import CricketDelivery
9     from src.core.schemas.pickleball import PickleballRally
10    from src.core.schemas.tabletennis import TableTennisRally
11
12    # Maps each sport to its annotation table and the column(s) that order segments
13    # chronologically. All child tables expose (start_time, end_time), so a
14    # single generic reader can serve every sport ? adding a new sport is one entry
15    # here plus its CREATE TABLE, with no new query code.
16    SEGMENT_TABLES = {
17        SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18        SportType.TENNIS: ("tennis_points", "point_number"),
19        SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20        SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21        SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22    }
23
24
25    class SportsDatabase:
26        def __init__(self, db_path: str):
27            self.db_path = db_path
28            # Ensure target directory exists
29            os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30            self._init_db()
31
32        @contextmanager
33        def _get_connection(self):
34            """Yield a SQLite connection and always close it.
35

```

```

36         Used as ``with self._get_connection() as conn:``. A plain
37         ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38         transaction but does NOT close the connection, which leaks handles and
39         raises ResourceWarnings. This wrapper guarantees the connection is closed
40         on exit; writers still commit explicitly inside the block.
41         """
42         conn = sqlite3.connect(self.db_path)
43         conn.row_factory = sqlite3.Row
44         # SQLite disables foreign keys per-connection by default; enable so the
45         # ON DELETE CASCADE declarations on the child tables actually fire.
46         conn.execute("PRAGMA foreign_keys = ON")
47         try:
48             yield conn
49         finally:
50             conn.close()
51
52     def _init_db(self):
53         with self._get_connection() as conn:
54             cursor = conn.cursor()
55
56             # 1. Create common videos table
57             cursor.execute("""
58             CREATE TABLE IF NOT EXISTS videos (
59                 video_id TEXT PRIMARY KEY,
60                 sport TEXT NOT NULL,
61                 video_url TEXT,
62                 local_path TEXT,
63                 license_type TEXT NOT NULL,
64                 license_url TEXT,
65                 is_commercial INTEGER NOT NULL DEFAULT 0,
66                 status TEXT NOT NULL DEFAULT 'staging',
67                 match_name TEXT NOT NULL,
68                 fps REAL,
69                 resolution TEXT
70             )
71             """)
72
73             # 2. Create badminton_rallies table
74             cursor.execute("""
75             CREATE TABLE IF NOT EXISTS badminton_rallies (
76                 video_id TEXT NOT NULL,
77                 rally_number INTEGER NOT NULL,
78                 start_time REAL NOT NULL,
79                 end_time REAL NOT NULL,
80                 score_before TEXT,
81                 score_after TEXT,
82                 shots_count INTEGER,
83                 ending_reason TEXT,
84                 last_shot_outcome TEXT,
85                 PRIMARY KEY (video_id, rally_number),
86                 FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
87             )
88             """)
89
90             # 3. Create tennis_points table
91             cursor.execute("""
92             CREATE TABLE IF NOT EXISTS tennis_points (
93                 video_id TEXT NOT NULL,
94                 point_number INTEGER NOT NULL,
95                 start_time REAL NOT NULL,
96                 end_time REAL NOT NULL,
97                 set_score TEXT,
98                 game_score TEXT,
99                 server TEXT,
100                 ending_type TEXT,

```

```

101         PRIMARY KEY (video_id, point_number),
102         FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
103     )
104     """)
105
106 # 4. Create cricket_deliveries table
107 cursor.execute("""
108 CREATE TABLE IF NOT EXISTS cricket_deliveries (
109     video_id TEXT NOT NULL,
110     over_num INTEGER NOT NULL,
111     ball_num INTEGER NOT NULL,
112     start_time REAL NOT NULL,
113     end_time REAL NOT NULL,
114     batsman TEXT,
115     bowler TEXT,
116     runs INTEGER,
117     wicket INTEGER DEFAULT 0,
118     extras INTEGER DEFAULT 0,
119     PRIMARY KEY (video_id, over_num, ball_num),
120     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
121 )
122 """)
123
124 # 5. Create pickleball_rallies table
125 cursor.execute("""
126 CREATE TABLE IF NOT EXISTS pickleball_rallies (
127     video_id TEXT NOT NULL,
128     rally_number INTEGER NOT NULL,
129     start_time REAL NOT NULL,
130     end_time REAL NOT NULL,
131     score_before TEXT,
132     score_after TEXT,
133     shots_count INTEGER,
134     ending_reason TEXT,
135     last_shot_outcome TEXT,
136     PRIMARY KEY (video_id, rally_number),
137     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
138 )
139 """)
140
141 # 6. Create tabletennis_rallies table
142 cursor.execute("""
143 CREATE TABLE IF NOT EXISTS tabletennis_rallies (
144     video_id TEXT NOT NULL,
145     rally_number INTEGER NOT NULL,
146     start_time REAL NOT NULL,
147     end_time REAL NOT NULL,
148     score_before TEXT,
149     score_after TEXT,
150     shots_count INTEGER,
151     ending_reason TEXT,
152     last_shot_outcome TEXT,
153     PRIMARY KEY (video_id, rally_number),
154     FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
155 )
156 """)
157
158 # Migration: add last_shot_outcome to rally tables that predate the
159 # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
160 # do it explicitly and idempotently (skip if the column is already there).
161 for tbl in ("badminton_rallies", "pickleball_rallies",
"tabletennis_rallies"):
162     existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl}))").fetchall()]
163     if "last_shot_outcome" not in existing:

```



```

225         """ , (
226             rally.video_id,
227             rally.rally_number,
228             rally.start_time,
229             rally.end_time,
230             rally.score_before,
231             rally.score_after,
232             rally.shots_count,
233             rally.ending_reason,
234             rally.last_shot_outcome
235         ))
236     conn.commit()
237     return len(rallies)
238
239     def insert_tennis_points(self, points: List[TennisPoint]) -> int:
240         if not points:
241             return 0
242         with self._get_connection() as conn:
243             cursor = conn.cursor()
244             video_id = points[0].video_id
245             cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
246
247             for pt in points:
248                 cursor.execute("""
249                     INSERT INTO tennis_points (
250                         video_id, point_number, start_time, end_time,

```

24. assistant (2026-06-08T08:39:27.404Z)

Now let me look at the CVAT rally schema and any annotation-related documentation:

25. user (2026-06-08T08:39:28.148Z)

C:\Users\avidu\Projects\sports-data-collector/docs/annotation
C:\Users\avidu\Projects\sports-data-collector/docs/annotation/ROORA_BRIEF.md

26. user (2026-06-08T08:39:30.136Z)

```

1      # Roora Annotation Brief ? Rally Golden Set
2
3      **Project:** racket-sport rally segmentation golden / regression dataset
4      **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles _and_
doubles**)
5      **Companion doc:** [ `ANNOTATION_GUIDE.md` ] (./ANNOTATION_GUIDE.md)
6
7      > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8      > imports directly via `python -m src.cli.curate import-local`.
9
10     ## 0. What we need in one sentence
11     For each provided match video, produce **one CSV file** listing **every rally** with its
12     start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14     ## 1. Deliverable format (canonical)
15     One CSV per video (UTF-8), named exactly `<video_id>.csv` (we supply the `video_id`).
16     One row per rally, chronological, header row required.
17
18     | column | meaning |
19     |---|---|
20     | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21     | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22     | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23     | `start_time` | decimal **seconds** (e.g. `72.40`). **Auto-filled** from `start_mmss`.
|

```

```

24 | `end_time` | decimal **seconds**. **Auto-filled** from `end_mmss`. |
25 | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26 | `ending_reason` | one of: `winner` \ | `forced_error` \ | `unforced_error` \ |
  | `service_fault` \ | `let` \ | `other` (**why** it ended) |
27 | `last_shot_outcome` | one of: `in` \ | `net` \ | `out` \ | `n_a` (**where** the last shot
  | landed; `n_a` for service_fault/let) |
28 | `score_before` | optional, `"A-B"`. Blank if not confidently readable. |
29 | `score_after` | optional, `"A-B"`. |
30 | `notes` | optional free text. **Required** when `ending_reason` = other`. |
31
32 **Times (mm:ss ? decimal seconds). Raters type only `start_mmss`/`end_mmss`. The
  template
33 Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
  row 2):
34
35 ```
36 =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
  VALUE(MID(B2,FIND(":",B2)+1,20))), VALUE(B2)))
37 ```
38
39 On **Download as CSV**, the computed decimal seconds are exported. (If your tool already
  shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
  columns blank.)
40
41 Example (badminton):
42 ```csv
43 rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,last_shot
  _outcome,score_before,score_after,notes
44 1,0:12.50,0:25.00,12.50,25.00,9,winner,in,0-0,1-0,
45 2,0:45.00,0:47.30,45.00,47.30,1,service_fault,n_a,1-0,1-1,serve into net
46 3,1:01.20,1:18.85,61.20,78.85,14,unforced_error,out,1-1,2-1,final clear sailed long (no
  pressure)
47 ```
48
49 **Critical format rules**
50
51 - `start_time`/`end_time` that reach us **must be decimal seconds** (the formula
  guarantees this).
52
53 - Times are relative to the **start of the file we provide**. Do **not** trim, clip, or
  re-encode the video ? our pipeline identifies a video by its file checksum.
54
55 - `end_time` > `start_time` for every row.
56
57 - No overlaps: `end_time[N]` <= `start_time[N+1]`.
58
59 - Everything between one rally's `end_time` and the next rally's `start_time` is **dead
  time** and gets **no row**.
60
61 **2. The three non-negotiable rules (full detail + pictures in the Guide)
62
63 1. **Ignore all dead time.** Only label active play.
64
65 2. **Include every rally ? exclude none.** Every served point, including **service
  faults**,
66
67 1?2 shot rallies, and **lets/replays**. A missed rally is the worst error in this
  project.
68
69 **Exception: warm-up / knock-up is NOT a rally** ? pre-match and between-games warm-
  up looks
70
71 like rallying but isn't scored play, so **don't label it**. Tell it apart by: no real
  serve,
72
73 cooperative (not competitive) hitting, the score not progressing, often several
  shuttles/balls
74
75 in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
  it.
76
77 3. **A shot = one contact** with the racket/paddle/bat. Serve = shot 1. Bounces and net-
  clips are not shots.
78
79 **3. Scope & phasing
80
81 **Phase A ? Pilot (this engagement):** 3 videos (1 badminton + 1 pickleball + 1 table
  tennis),
82
83 each < 3 min` with ~10?12 rallies. This is the **paid calibration round**; we review

```



```

jointly and
71     lock the Guide before scale-up.
72
73     **Phase B ? Scale (after a successful pilot):** ~12?15 videos **per sport** (~36?45
total), each
74     ~30 min. Same format, rules, and template.
75
76     > **Doubles are included** across the program. Rally rules are identical to singles; the
77     > badminton- and pickleball-specific doubles notes are in the Guide.
78
79     ## 4. Quality bar (acceptance criteria)
80     Automated validation (no overlaps, positive durations, schema-valid) runs on every file,
then a
81     ~20% human spot-check. A file is returned for rework (in scope) if it misses:
82     - **Completeness:** every rally present; zero missed rallies; zero dead-time rows.
83     - **Boundaries:** `start_time`/`end_time` within **±0.3 s** of true serve-contact /
landing.
84     - **shots_count:** exact for rallies ? 15 shots; ±1 allowed for longer/faster rallies.
85     - **ending_reason:** one of the six allowed values, per the Guide's decision tree.
86     - **Format:** decimal seconds present, valid columns, non-overlapping, chronological.
87
88     ## 5. Per-sport quick reference (full version + examples in the Guide)
89     **Badminton (singles & doubles)** ? start: racket?shuttle serve contact · end: shuttle
floors / fault · shot: each racket?shuttle contact.
90     **Pickleball (singles & doubles)** ? start: paddle?ball serve contact · end: double
bounce / out / net / fault · shot: each paddle?ball contact (floor bounces are **not** shots;
two-bounce rule is normal play).
91     **Table tennis** ? start: racket?ball serve strike (not the toss) · end: failed legal
return · shot: each racket?ball contact (table bounces are **not** shots). Net-cord serve =
`let`.
92
93     ## 6. ending_reason values (decision tree in the Guide)
94     This vocabulary attributes **why** the point ended; it is shared across all
95     net-separated sports (badminton, pickleball, table tennis, tennis). Record **where**
96     the last shot physically landed in the separate **`last_shot_outcome`** column
97     (`in` \| `net` \| `out` \| `n_a`) ? e.g. a forced error that flew long is
98     `ending_reason=forced_error, last_shot_outcome=out`.
99
100    | value | meaning |
101    |---|---|
102    | `winner` | last shot landed **in** and the opponent made no legal contact (a clean
winner). |
103    | `forced_error` | the losing player erred (netted / out / missed) **under pressure** ?
stretched, wrong-footed, or rushed by the opponent's strong shot. |
104    | `unforced_error` | the losing player erred on a **routine, unpressured** ball they had
time and position to make. |
105    | `service_fault` | the serve itself failed / was illegal. |
106    | `let` | the point was replayed. |
107    | `other` | anything else / cannot tell. **Must** add a note. |
108
109    ## 7. Recommended capture method
110    Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
111    **step one frame** forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
112    (Label Studio / CVAT). The deliverable is the CSV in the schema above.
113    `seconds = frame_number / fps` (we provide fps per video).
114
115    ## 8. Logistics & contacts
116    - **TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
117    - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[INTAKE_MANIFEST_TEMPLATE.csv](./INTAKE_MANIFEST_TEMPLATE.csv). **TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
118    - **TODO(avidullu): paste the Drive folder link** where Roora uploads completed

```

```

`<video_id>.csv` files.
119 - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
**TODO(avidullu): create it and paste the link.**
120 - **TODO(avidullu): add the escalation channel** (email / WhatsApp group) for edge-case
questions.
121 - NDA before any footage is shared ? template [ `NDA_TEMPLATE.md` ] (./NDA_TEMPLATE.md).
**TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
122 - Roora contact on file: `info@rooragrp.com` (HSR Layout, Bangalore).
123
124 ## 9. Pricing request (please quote)
125 1. Price **per rally** and **per minute** of footage.
126 2. **Pilot** price (3 short videos, ~30?36 rallies) as a paid calibration round.
127 3. Indicative **scale** price for ~36?45 longer videos.
128 4. Turnaround for the pilot and throughput (rallies/day) at scale.
129 5. QA process and rework policy.
130 6. Whether annotators have racket-sport familiarity (helpful, not required).
131 7. Confirmation you can deliver the CSV schema above.
132

```

27. assistant (2026-06-08T08:39:32.781Z)

Perfect! Now let me check if there are any other relevant vendor-specific CSVs or related schemas:

28. user (2026-06-08T08:39:34.829Z)

```

Found 10 files limit: 10
backend\eval\eval_partial_labels.py
docs\README.md
docs\PLATFORM_ARCHITECTURE.md
docs\COMMERCIALIZATION.md
tests\test_config.py
tests\test_rally_slicer.py
run_process_and_stitch.py
docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
docs\archives\decisions\DECISIONS.md
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md

```

29. user (2026-06-08T08:39:40.005Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer-
khelacharya/.git/logs/refs/heads/docs/golden-set-vendoring
C:\Users\avidu\Projects\badminton-highlight-indexer-
khelacharya/.git/logs/refs/remotes/origin/docs/golden-set-vendoring
C:\Users\avidu\Projects\badminton-highlight-indexer-khelacharya/.git/refs/heads/docs/golden-set-
vendoring
C:\Users\avidu\Projects\badminton-highlight-indexer-
khelacharya/.git/refs/remotes/origin/docs/golden-set-vendoring
C:\Users\avidu\Projects\munem/.venv\Lib/site-packages/pip/_vendor
C:\Users\avidu\Projects\munem/.venv\Lib/site-packages/pip/_vendor/vendor.txt
C:\Users\avidu\Projects\munem/.venv\Lib/site-packages/pip-26.1.1.dist-
info/licenses/src/pip/_vendor
C:\Users\avidu\Projects\rally-annotator
C:\Users\avidu\Projects\rally-annotator\vlc\rally_annotator.lua

```

30. assistant (2026-06-08T08:39:41.978Z)

Let me check the rally-annotator project for any vendor-specific or annotator schemas:

31. user (2026-06-08T08:39:42.585Z)

```

C:\Users\avidu\Projects\rally-annotator\CHANGELOG.md
C:\Users\avidu\Projects\rally-annotator\docs\CSV_FORMAT.md
C:\Users\avidu\Projects\rally-annotator\docs\ENDING_REASONS.md

```

32. user (2026-06-08T08:39:44.666Z)

```
1      # Output CSV format
2
3      One row per rally. Header is written once when the file is created.
4
5      ```
6      rally_number,start_time,end_time,ending_reason,sport
7      1,8.800,11.500,winner,badminton
8      2,24.389,46.589,unforced_error,badminton
9      3,49.183,54.683,let,badminton
10     ```
11
12     | Column | Type | Notes |
13     |---|---|---|
14     | `rally_number` | int | 1-based, monotonic; continues across re-opening the same CSV |
15     | `start_time` | float | rally start, **decimal seconds** from the video start |
16     | `end_time` | float | rally end, decimal seconds (always `> start_time`) |
17     | `ending_reason` | enum | `winner` / `forced_error` / `unforced_error` /
`service_fault` / `let` / `other` ? see the decision guide in
[ENDING_REASONS.md](ENDING_REASONS.md) |
18     | `sport` | enum | `badminton` / `tennis` / `table_tennis` / `pickleball` / `padel` |
19
20     Notes:
21     - **Decimal seconds**, not `mm:ss` ? downstream loaders parse floats.
22     - Rows are appended live; **Undo last** truncates the last row (one level).
23     - Consumers should read by **column name** (the header is stable but column *order* may
evolve; extra
24     columns may be added later). Most CSV `DictReader`-style loaders already tolerate
extra columns.
25
26     ## Ingesting the labels
27
28     This CSV is intentionally compatible with common rally-segmentation tooling that reads
29     `rally_number,start_time,end_time` by header name and drops rows where `end <= start`:
30
31     ```bash
32     # Generic: any tool that takes a labels CSV of start/end seconds.
33
34     # Example ? score a detector against these labels, or slice clips by them
35     # (badminton-highlight-indexer loaders read --labels <this.csv> directly):
36     python -m backend.eval.calibrate_wasb --trajectory traj.csv --labels match.rallies.csv
--fps 59.94 --frame-width 1920
37     python -m backend.tools.rally_slicer --video match.mp4 --labels match.rallies.csv
--rallies all
38
39     # Example ? register as curated golden data in sports-data-collector:
40     python -m src.cli.curate import-local --sport badminton --video-path match.mp4
--annotations-path match.rallies.csv
41     ```
42
43     The `sport` column is self-documenting; tools that take a `--sport` flag can ignore it.
44
```

33. user (2026-06-08T08:39:44.767Z)

```
1      # Ending reasons ? what they mean & how to label them
2
3      The `ending_reason` column records **why the rally (point) ended**, judged from the
4      **last shot**. There are six values, shared across all the net-separated racquet sports
5      this tool supports (badminton, tennis, table tennis, pickleball, padel):
6
7      ```
```

```

8      winner | forced_error | unforced_error | service_fault | let | other
9      ```
10
11      **One core principle:** every reason **except `winner`** is charged to the player/side
12      that **LOST** the rally. `winner` is the only one that credits the side that **won** it.
13
14      This guide is written for a human rater clicking through a match. Where a rule comes
15      from the laws of the sport it is marked **[rule]**; where it is an analytics *convention* (a
16      judgement call that charters agree to apply consistently) it is marked **[convention]**.
17
18      ---
19
20      ## The decision procedure (top to bottom ? stop at the first match)
21
22      1. **Was the rally replayed under the rules, with no point scored?**
23          (e.g. a tennis/padel serve that clips the net and lands in the correct box; a table-
24      tennis
25      serve that touches the net and is otherwise good; outside interference; receiver not
26      ready.)
27          ? **`let`**. *A net-cord during open rally play that lands in is NOT a let ? keep
28      going.*
29
30      2. **Did the point end on the SERVE because of a server fault?**
31          (serve into the net, serve outside the correct service court, illegal service action
32      / foot
33      fault, or a tennis/padel **double fault**.)
34          ? **`service_fault`**.
35
36      3. **Did the winning side's last shot land IN and go unreturned** ? the opponent
37      couldn't
38      reach it, or only waved at it without meaningful contact? (A clean untouched serve /
39      **ace**
40      counts here.) ? **`winner`**.
41
42      4. **Otherwise the rally ended on a MISS by the loser** (ball/shuttle **out**, **into
43      the net**,
44      or not legally returned). Now judge **pressure**:
45
46          > **The pressure test:** Did the loser have **both enough time AND court position**
47      to make a
48          > routine return, given the opponent's *immediately preceding* shot (its placement,
49      pace,
50          > power, spin, depth, angle)? Ask: *"Would a typical competent player at this level
51      be
52          > expected to make that shot?"* **[convention]**
53
54          - **Yes** ? had time and position, missed a routine ball ? **`unforced_error`**.
55          - **No** ? time or position was taken away; the miss was induced ?
56      **`forced_error`**.
57
58      5. **Can't judge it, or it ended off-court** (occluded footage, injury/retirement,
59      code/point
60      penalty, hindrance, equipment failure) ? **`other`**. Use sparingly.
61
62      **Tie-breaker [convention]:** when you genuinely can't tell forced from unforced,
63      **default to
64      `unforced_error`**. Only mark `forced_error` when you can *point to the specific
65      pressure* (the
66      wide/fast/deep/spinny ball that removed the player's time or position). This keeps
67      `forced_error`
68      a positively-evidenced label and stops it from inflating. The forced/unforced boundary
69      is a real
70      *continuum* ? charters disagree on borderline calls, so fix the default in writing for
71      your team.

```

```

55
56     ---
57
58     ## The six reasons, defined
59
60     ### `winner`
61     The point-ending shot landed **in** and the opponent did **not** make a meaningful
return
62     (couldn't reach it, or only waved). Credit to the side that won the rally. A clean
untouched
63     legal serve (an **ace**) is conventionally a `winner`, not a `service_fault`.
64     *Examples:* a smash that bounces untouched; a deceptive drop the opponent never reaches;
a
65     passing shot down the line.
66     *Not a winner:* if the opponent reaches it but mishits **under pressure**, that is
**their**
67     `forced_error`, not your `winner`.
68
69     ### `forced_error`
70     A miss by the **loser** (out, into the net, or not legally returned) where the
opponent's
71     previous shot applied enough pressure ? placement, pace, power, spin, depth, angle ?
that the
72     player did **not** have time and position for a routine return. They were made to
miss.
73     *Examples:* a wide, heavy serve return netted while lunging at full stretch; a hard
smash dug up
74     but floated long because the player was off-balance and rushed.
75
76     ### `unforced_error`
77     A miss by the **loser** on a shot they had **time AND position** to make routinely, with
little
78     or no pressure. They gave it away.
79     *Examples:* a comfortable mid-court ball drilled into the net; a relaxed clear pushed
past the
80     back line; a sitter smashed long.
81
82     ### `service_fault`
83     A fault by the **server on the serve itself** that loses the point or the serve ? judged
against
84     the sport's service rules, not general rally play. Includes: serve into the net, serve
out of
85     the correct service court/box, illegal service action (e.g. badminton contact above 1.15
m,
86     ITTF illegal toss, pickleball illegal motion), foot fault, and a tennis/padel **double
fault**.
87     *Note:* a **first**-serve fault followed by a good second serve is **not** charged here
? only the
88     serve that actually loses the point is. A clean ace is a `winner`, not a
`service_fault`.
89
90     ### `let`
91     A **replay** defined by the laws, where **nobody is charged** a point ? the rally
"didn't count":
92     a tennis/padel service net-cord that still lands in the correct box (replayed, unlimited
times);
93     a table-tennis serve that touches the net and is otherwise correct; an outside
interruption; the
94     receiver not ready. Use **only** when the rule says replay.
95     *Caution:* a net-cord during a **rally** (not the serve) that lands in is live ? **not**
a `let`.
96
97     ### `other`
98     Catch-all for endings that fit none of the above: the outcome is genuinely
unclear/occluded on

```

```

99      video; or the point ended administratively (injury retirement, code/point penalty,
hindrance
100      call, equipment failure) rather than on a struck shot. Prefer a specific category
whenever the
101      footage supports it.
102
103      ---
104
105      ## The cases people ask about most
106
107      These are the calls that trip raters up ? resolved precisely.
108
109      ### A. The ball/shuttle lands **OUT** (past the baseline or outside the sidelines)
110      This is **always the hitter's error** ? the side that **lost** the rally. The opponent
merely let
111      it sail (or it cleared the line untouched), so it is **never** the opponent's `winner`.
Decide
112      `forced` vs `unforced` purely by **pressure on the hitter** at the moment they struck it
? look at
113      the opponent's *previous* shot, not the out ball itself:
114      - Rushed, stretched, off-balance, or handling heavy pace/spin/depth ?
**`forced_error`**.
115      - Set and balanced, with time on a routine ball, and still sprayed it ?
**`unforced_error`**.
116
117      "Beyond the baseline" vs "outside the sideline" doesn't change the logic ? both are
*out*, both are
118      the hitter's miss. (In tennis and badminton the **line is in** ; this applies only when
it lands
119      fully outside.) **So "out" is not automatically `forced`** ? that label needs visible
pressure;
120      otherwise it's `unforced`.
121
122      ### B. The ball/shuttle goes **INTO the net** (fails to cross)
123      Same as an "out" ball ? it's the hitter's error, and the **pressure test** decides:
`forced_error`
124      if they were rushed/stretched, `unforced_error` if they dumped a routine ball.
125      **Exception:** if the failure-to-cross happens **on the serve**, it is a
**`service_fault`** (in
126      tennis/padel, a missed second serve = double fault), not a rally error ? the rally never
legally
127      began.
128
129      ### C. The **net-cord / net tape** (ball or shuttle clips the top)
130      - **During a rally:** it is **live and good** in all of these sports. Play continues, so
the rally
131      ends on whatever happens **next**: if the opponent can't reach the dribbler ? it's a
`winner`
132      (a "net-cord winner"); if they scramble and then miss ? judge *that* miss as
forced/unforced.
133      A rally net-cord is **never** a `let`.
134      - **On the serve ? sports differ:**
135      | Sport | Serve clips the net and? |
136      |---|---|
137      | **Tennis / Padel** | ?lands in the correct box ? **`let`** (replay). Lands outside ?
`service_fault`. |
138      | **Table tennis** (ITTF) | ?is otherwise correct ? **`let`** (serve replayed). |
139      | **Badminton** (BWF) | ?passes over and lands in the service court ? **live, play
on** (no service let). But a serve shuttle **caught/suspended on the net** is a
**`service_fault`** (BWF Law 13.2), *not* a let. |
140      | **Pickleball** (2026 USAP) | ?lands in the correct court ? **live, play on** (the
old service-let was removed). Lands out ? `service_fault`. |
141
142      ---
143

```

```

144     ## Sport-specific notes
145
146     - **Tennis (ITF):** double fault ? `service_fault`; service net-cord into the box ?
`let`; lines
147     are **in**; a clean ace ? `winner`.
148     - **Badminton (BWF):** service contact must be with the whole shuttle below **1.15 m**
from the
149     court surface (Law 9.1.6; the older "above the waist" wording is the alternative trial
law). A
150     shuttle landing outside the lines or failing to pass the net is a fault; a **serve
caught on the
151     net is a fault** (`service_fault`), while a mid-rally suspended shuttle (Law 14.2.3,
only *after*
152     the serve is returned) is a `let`. Lines are **in**.
153     - **Table tennis (ITTF):** a serve touching the net assembly but otherwise correct is a
**`let`**
154     (replayed); in a rally, a ball that touches the net and still lands on the opponent's
court is a
155     **good return**.
156     - **Pickleball (USAP 2026):** **no** service let ? a net-clip that lands in the correct
court is
157     live; illegal serve motion / net-clip-out ? `service_fault`.
158     - **Padel (FIP):** like tennis for the serve net-cord `let`. Wall play means a rally
ball off the
159     glass after one bounce is still live. (One edge case ? a let-bound serve that touches
the net and
160     then the side **wire mesh** before its 2nd bounce ? is finicky; verify against current
FIP text.)
161
162     ---
163
164     ## Consistency tips
165
166     - The forced/unforced split is a **judgement call on a continuum**. Pick the default
167     (`unforced` when unsure) and apply it the same way every time ? consistency matters
more than
168     any single borderline call.
169     - If you're building a rating *team* or a regression set, consider logging a confidence
note on
170     borderline forced/unforced calls so they can be audited or re-bucketed later.
171     - These six values are deliberately a small, shared vocabulary. Keep to them (don't
invent new
172     spellings) so downstream aggregation stays clean.
173
174     ### Mapping to tennis charting codes (optional)
175     If you cross-reference the [Match Charting
Project](https://github.com/JeffSackmann/tennis_MatchChartingProject):
176     `winner = *`, `forced_error = #`, `unforced_error = @` (with location tags `n` = net,
`w` = wide,
177     `d` = deep, `x` = wide+deep). These are charting conventions layered on top of the
official rules.
178
179     ---
180
181     ## Sources
182
183     - Match Charting Project ? quick-start & codes:
<https://www.tennisabstract.com/blog/2015/09/23/the-match-charting-project-quick-start-guide/>
184     - "The continuum of errors" (forced vs unforced is a continuum):
<http://www.tennisabstract.com/blog/2017/01/13/the-continuum-of-errors/>
185     - Leo Levin's pressure definition of unforced error:
<https://www.tennis.com/news/articles/what-is-an-unforced-error-a-meditation-on-the-tennis-
player-s-least-favorite-stat>
186     - ITF Rules of Tennis: <https://www.itftennis.com/en/about-us/governance/rules-and-
regulations/>

```

187 - BWF Laws of Badminton: <<https://www.worldbadminton.com/rules/>>
188 - ITTF Table Tennis rules (Law 2.7 / 2.9): <<https://www.ittf.com/handbook/>>
189 - USA Pickleball rulebook: <<https://usapickleball.org/what-is-pickleball/official-rules/>>
190 - FIP Rules of Padel: <<https://www.padelfip.com/rules-of-padel/>>
191

34. assistant (2026-06-08T08:39:48.177Z)

Excellent. Now let me check the badminton-highlight-indexer to see how it reads evaluation/vendor CSVs:

35. user (2026-06-08T08:39:48.854Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.git/logs/refs/remotes/origin/docs/commercial-readiness-review-and-plan
C:\Users\avidu\Projects\badminton-highlight-indexer/.git/refs/remotes/origin/docs/commercial-readiness-review-and-plan
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/eval/__pycache__/eval_partial_labels.cpython-313.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/__pycache__/test_eval_partial_labels.cpython-313-pytest-9.0.3.pyc
```

36. user (2026-06-08T08:39:51.000Z)

```
1      """Evaluate the rally pipeline against PARTIAL human golden labels.
2
3      The VLC rally-annotator produces a human-rated CSV (rally_number,start_time,end_time,
4      ending_reason,sport). While a video is only partially tagged, the labels cover a
5      PREFIX of the video ? every rally up to some point is marked, and everything after is
6      simply un-watched (not "no rally there"). Scoring naively against such labels is wrong:
7      the detector's *correct* rallies in the un-labeled tail would be counted as false
8      positives and crush precision.
9
10     This harness fixes that by restricting the evaluation to the LABELED WINDOW
11     [window_start, window_end] (window_end defaults to the last labeled rally's end) and
12     dropping predicted segments that fall outside it. Within that window the human labels
13     ARE complete, so precision/recall/F1 are honest.
14
15     It reuses everything from the existing eval stack (no new windowing/metrics/gate logic):
16     - calibrate_wasb.make_predict_fn / load_golden_gts / BASELINE
17     - pipeline.detectors.rally_gate.apply_gate (net-crossing "gate A")
18     - eval.metrics.match_intervals / score
19     - export_wasb_segments.build_comparison / write_comparison_csv (per-rally detail CSV)
20
21     Run:
22     python -m backend.eval.eval_partial_labels \
23         --trajectory C:/Users/avidu/AppData/Local/Temp/adarsh_avi_traj.csv \
24         --labels "C:/Users/avidu/Projects/Annotation Setup/Collect/Adarsh and
25     Avi.rallies.csv" \
26         --fps 59.94 --frame-width 1920
27
28     """
29     from __future__ import annotations
30
31     import argparse
32     import os.path as osp
33     from typing import List, Optional, Tuple
34
35     from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
36     from backend.eval.metrics import score
37     from backend.eval.export_wasb_segments import build_comparison, write_comparison_csv
38     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```



```

37     from backend.pipeline.detectors import rally_gate
38
39     Interval = Tuple[float, float]
40     Config = Tuple[float, float, float]
41
42     # Operating points to compare. The tuned config wins on its fit-video but over-segments
43     # and does not transfer; baseline (+ net-crossing gate) is the stronger general point.
44     TUNED: Config = (0.05, 1.0, 1.0)
45
46
47     def restrict_to_window(intervals: List[Interval], w0: float, w1: float) ->
List[Interval]:
48         """Keep intervals that overlap the labeled window [w0, w1].
49
50         A predicted segment is evaluated iff it overlaps the labeled region; segments wholly
51         in the un-labeled tail (start >= w1) or before the window (end <= w0) are dropped so
52         they are neither rewarded nor penalized. GTs are assumed already inside the window.
53         """
54         return [(s, e) for (s, e) in intervals if e > w0 and s < w1]
55
56
57     def _eval_point(predict_fn, cfg: Config, points, fps: float, gts: List[Interval],
58                     window: Tuple[float, float], gate_crossings: int, iou: float) -> dict:
59         """Score one (config, gate) operating point inside the labeled window."""
60         preds = predict_fn(cfg)
61         if gate_crossings > 0 and points:
62             preds = rally_gate.apply_gate(points, preds, fps, min_crossings=gate_crossings)
63             preds = restrict_to_window(preds, *window)
64             sc = score(preds, gts, iou)
65             return {"cfg": cfg, "gate": gate_crossings, "n_pred": len(preds), "score": sc,
"preds": preds}
66
67
68     def run(trajecory_csv: str, labels_csv: str, fps: float, frame_width: float,
69            window_start: float = 0.0, window_end: Optional[float] = None,
70            gate_crossings: int = 2, iou: float = 0.5,
71            comparison_out: Optional[str] = None) -> dict:
72         gts_all = load_golden_gts(labels_csv)
73         if not gts_all:
74             raise SystemExit(f"no usable human rallies in {labels_csv}")
75         if window_end is None:
76             window_end = max(e for _, e in gts_all)
77         window = (window_start, window_end)
78         gts = restrict_to_window(gts_all, *window)
79
80         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
81         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
82         traj_end = (max(p.frame for p in points) / fps) if points else 0.0
83
84         print("=== Pipeline vs PARTIAL human labels ===")
85         print(f"labels: {labels_csv}")
86         print(f" {len(gts_all)} human rallies; labeled window [{window_start:.1f}s,
{window_end:.1f}s] "
87               f"-> {len(gts)} rallies in-window")
88         print(f"trajectory covers ~0-{traj_end:.1f}s "
89               f"({'covers the window' if traj_end >= window_end - 1 else 'WARNING: shorter
than window'})")
90         print(f"IoU>={iou}; net-crossing gate K={gate_crossings}\n")
91
92         # Compare the standard operating points: baseline / baseline+gate / tuned /
tuned+gate.
93         points_for_gate = points
94         candidates = [
95             ("baseline", BASELINE, 0),
96             (f"baseline+gate{gate_crossings}", BASELINE, gate_crossings),

```

```

97         ("tuned", TUNED, 0),
98         (f"tuned+gate{gate_crossings}", TUNED, gate_crossings),
99     ]
100     results = []
101     header = f"{'operating point':<18} {'pred':>4} {'TP':>3} {'FN':>3} {'FP':>3} " \
102             f"{'prec':>6} {'recall':>6} {'F1':>6} {'mIoU':>6} {'sErr':>6} {'eErr':>6}"
103     print(header)
104     print("-" * len(header))
105     for name, cfg, gate in candidates:
106         r = _eval_point(predict_fn, cfg, points_for_gate, fps, gts, window, gate, iou)
107         sc = r["score"]
108         r["name"] = name
109         results.append(r)
110         print(f"{name:<18} {r['n_pred']:>4} {sc.true_positives:>3}
{sc.false_negatives:>3} "
111               f"{sc.false_positives:>3} {sc.precision:>6.3f} {sc.recall:>6.3f}
{sc.f1:>6.3f} "
112               f"{sc.mean_iou:>6.3f} {sc.mean_start_error:>6.2f}
{sc.mean_end_error:>6.2f}")
113
114     best = max(results, key=lambda r: r["score"].f1)
115     print(f"\nbest operating point by F1: {best['name']} (F1 {best['score'].f1:.3f})")
116
117     # Per-rally detail for the best point: which human rallies matched/missed + extra
    fires.
118     comp = build_comparison(best["preds"], gts, iou)
119     comp_out = comparison_out or osp.join("output", "adarsh_avi_vs_human.csv")
120     write_comparison_csv(comp_out, comp)
121     print(f"per-rally detail ({best['name']}) -> {comp_out}")
122
123     return {
124         "window": window, "n_human_in_window": len(gts),
125         "results": [{"name": r["name"], "cfg": r["cfg"], "gate": r["gate"],
126                     "n_pred": r["n_pred"], **r["score"].as_dict()} for r in results],
127         "best": best["name"], "comparison_csv": comp_out,
128     }
129
130
131     def main() -> None:
132         ap = argparse.ArgumentParser(description=__doc__)
133         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
134         ap.add_argument("--labels", required=True, help="human golden CSV
(rally_number,start_time,end_time,...)")
135         ap.add_argument("--fps", type=float, default=59.94)
136         ap.add_argument("--frame-width", type=float, default=1920.0)
137         ap.add_argument("--window-start", type=float, default=0.0)
138         ap.add_argument("--window-end", type=float, default=None,
139                         help="end of the labeled region (default: last labeled rally end)")
140         ap.add_argument("--gate", type=int, default=2, help="net-crossing gate K (0=off, 2
recommended)")
141         ap.add_argument("--iou", type=float, default=0.5)
142         ap.add_argument("--comparison-out", default=None)
143         args = ap.parse_args()
144         run(args.trajectory, args.labels, args.fps, args.frame_width,
145             window_start=args.window_start, window_end=args.window_end,
146             gate_crossings=args.gate, iou=args.iou, comparison_out=args.comparison_out)
147
148
149     if __name__ == "__main__":
150         main()

```

37. assistant (2026-06-08T08:39:53.310Z)

Now let me check the load_golden_gts function to see how it reads CSVs:

38. user (2026-06-08T08:39:54.034Z)

```
34: def load_golden_gts(csv_path: str) -> List[Interval]:
```

39. user (2026-06-08T08:39:56.706Z)

```
1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3      Wires the already-tested pieces together:
4      WASB trajectory CSV (Frame, Visibility, X, Y)
5      -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6      -> backend.eval.calibration (improvement_report / cross_validate)
7
8      Run (after a full-video WASB pass exists):
9      python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import csv
20      import argparse
21      from typing import List, Tuple, Callable
22
23      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
24      from backend.eval.calibration import (
25          select_best, improvement_report, cross_validate, evaluate,
26      )
27
28      Interval = Tuple[float, float]
29      Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
30
31      BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
32
33
34      def load_golden_gts(csv_path: str) -> List[Interval]:
35          """Rally [start,end] seconds from the golden CSV (start_time/end_time cols)."""
36          gts: List[Interval] = []
37          with open(csv_path, newline="", encoding="utf-8") as f:
38              reader = csv.DictReader(f)
39              reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
40              for row in reader:
41                  row = {(k or "").strip().lower(): v for k, v in row.items()}
42                  try:
43                      s = float((row.get("start_time") or "").strip())
44                      e = float((row.get("end_time") or "").strip())
45                  except (TypeError, ValueError):
46                      continue
47                  if e > s:
48                      gts.append((s, e))
49          return sorted(gts)
50
51
52      def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
53          """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
54          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
```

```

55
56     def predict_fn(cfg: Config) -> List[Interval]:
57         vt, mg, mwd = cfg
58         wins = TrackNetRunner.trajectory_to_action_windows(
59             points, fps=fps, frame_width=frame_width,
60             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
61         )
62         return [(w["start"], w["end"]) for w in wins]
63
64     return predict_fn
65
66
67     def default_grid() -> List[Config]:
68         return [(vt, mg, mwd)
69                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
70                 for mg in (1.0, 2.0, 3.0)
71                 for mwd in (1.0, 2.0, 3.0)]
72
73
74     def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
75             iou: float = 0.5) -> dict:
76         gts = load_golden_gts(labels_csv)
77         if not gts:
78             raise SystemExit(f"no usable golden rallies in {labels_csv}")
79         predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
80         grid = default_grid()
81
82         base = evaluate(predict_fn(BASELINE), gts, iou)
83         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
84 iou_threshold=iou)
85         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
86         (fit on full GT)
87         cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
88 out recall
89
90         print("=== WASB rally-segmentation calibration ===")
91         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
92         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
93 recall={base.recall:.3f} "
94 f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
95         print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
96         print(f" precision {fit['baseline']['precision']:.3f} ->
97 {fit['tuned']['precision']:.3f} "
98 f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
99         print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
100 f"({fit['f1_delta_pct']:+.1f}%)")
101         print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
102 f"f1={cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
103 f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
104         print("(For precision/F1 generalization, add a 2nd labelled video ->
105 leave_one_video_out.)")
106         return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}

```

40. assistant (2026-06-08T08:39:59.196Z)

Perfect! Now I have a complete picture. Let me create a structured summary by calling StructuredOutput with all the findings.

41. user (2026-06-08T08:40:40.157Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

42. user (2026-06-08T08:41:10.075Z)

Output does not match required schema: root: must have required property 'subsystem', root: must have required property 'summary', root: must have required property 'key_facts', root: must have required property 'contracts', root: must have required property 'gaps_or_mismatches', root: must have required property 'files'

43. assistant (2026-06-08T08:41:12.540Z)

Now let me provide the structured output with all required fields:

44. user (2026-06-08T08:41:47.068Z)

Structured output provided successfully

45. assistant (2026-06-08T08:41:55.649Z)

Perfect! I've completed a comprehensive analysis of the canonical rally schema, validation, and CSV I/O in the sports-data-collector project. Here's a summary of the key findings:

Canonical Rally Record

****Required fields:****

- `video\_id` (str)
- `rally\_number` (int, 1-based, chronological)
- `start\_time` (float, seconds, ≥ 0)
- `end\_time` (float, seconds, $> \text{start_time}$)

****Optional fields:****

- `score\_before` / `score\_after` (str, format "A-B")
- `shots\_count` (int)
- `ending\_reason` (str enum: winner | forced_error | unforced_error | service_fault | let | other)
- `last\_shot\_outcome` (str enum: in | net | out | n_a)

Validation Rules

- `start\_time  $\geq 0$ ` and `end\_time  $> \text{start\_time}$ ` (Pydantic validators)
- ****No temporal overlaps****: consecutive rallies must not overlap ($\text{end}[i] \leq \text{start}[i+1]$)
- Chronologically ordered by `rally\_number`

Canonical CSV Format

****On read**** (CVAT/import-local): 9-11 columns including rally_number, start_time/end_time (or start_mmss/end_mmss), shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes. Headers normalized to lowercase; field matching is lenient.

****On write**** (CVAT ? canonical): All 11 columns (rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes).

****On export**** (eval set): ****Only 2 columns**** (start, end in decimal seconds). All enrichment is dropped; this is intentional for generic interval-based evaluation.

Critical Gaps

1. ****ending_reason & last_shot_outcome**** are captured on import but ****NOT exported**** to eval sets. The indexer evaluation harness ignores these fields entirely.
2. ****shots_count**** is stored but not exported.
3. ****Fractional rally numbers**** (e.g., 1.1 for replays) are not supported.
4. ****Curation status/maturity**** (STAGING vs CURATED) is database-only, not in CSV.
5. ****Annotator/vendor metadata**** is not tracked per rally.
6. ****Round-trip fidelity is broken****: export ? re-import loses all enrichment fields.

The structured output above includes exact file paths, line anchors, and detailed contracts for all three systems.